

UNIVERSIDAD DE MURCIA
Facultad de Informática
Máster Universitario en Inteligencia Artificial

TRABAJO DE FIN DE MÁSTER

Evaluación de LLMs en la Generación de Ontologías a partir de Bases de Datos Genéticas

Comparativa entre tres modelos con varias estrategias de generación.

Autor

Francisco Javier Sáez González

Tutores

Jesualdo Tomás Fernández Breis

Ginés Almagro Hernández

Departamento de Informática y Sistemas

Curso académico 2025–2026

Convocatoria III

Murcia, julio de 2026

Resumen

Las ontologías constituyen la columna vertebral semántica de la biomedicina computacional: estructuran el conocimiento sobre genes, enfermedades y mecanismos regulatorios en representaciones formales que permiten razonamiento automático, integración de datos y descubrimiento de conocimiento. Sin embargo, su construcción manual exige un esfuerzo experto considerable —semanas de trabajo por dominio— y produce artefactos difícilmente reproducibles y costosos de mantener. Esta tensión entre la necesidad creciente de ontologías de dominio y el coste de producirlas manualmente ha abierto una línea de investigación activa en automatización ontológica.

En este contexto, los modelos de lenguaje grandes (LLM) han emergido como candidatos naturales para reducir ese coste: generan texto estructurado en formatos estándar, comprenden vocabulario técnico especializado y pueden condicionar su salida mediante estrategias de prompting, recuperación de contexto y arquitecturas multi-agente. No obstante, la literatura carece de evaluaciones sistemáticas y reproducibles que midan su capacidad real sobre bases de datos biomédicas reales, con métricas que cubran simultáneamente la corrección sintáctica, la calidad estructural normativa y la fidelidad funcional al dominio.

Este trabajo diseña y ejecuta un banco experimental reproducible para evaluar cuatro estrategias de generación —prompt directo (E1), vocabulario controlado (E2), RAG semántico (E3) y pipeline multi-agente OntoGenix (E4)— aplicadas de forma cruzada a tres modelos LLM (gpt-4o, Llama 3.1 8B y Qwen 2.5 Coder 7B) sobre cuatro bases de datos del dominio cis-regulatorio humano (FANTOM5, dbSUPER, HACER y DiseaseEnhancer). La evaluación combina validación sintáctica, métricas OQuaRE y preguntas de competencia.

Los resultados muestran que la pipeline multi-agente produce las ontologías estructuralmente más ricas pero recibe puntuaciones normativas intermedias por sobre-estructuración; el RAG semántico maximiza la riqueza relacional con modelos grandes y requiere calibración del retrieval en modelos compactos. El comportamiento es fuertemente dependiente del modelo: no existe una estrategia con ranking estable entre los tres LLM evaluados.

Palabras Clave

Ontologías biomédicas, modelos de lenguaje grandes, generación automática de ontologías, RAG semantic, ingeniería ontológica, OntoGenix, dominio cis-regulatorio, preguntas de competencia.

Abstract

Ontologies constitute the semantic backbone of computational biomedicine: they structure knowledge about genes, diseases, and regulatory mechanisms into formal representations that enable automated reasoning, data integration, and knowledge discovery. However, their manual construction requires considerable expert effort—weeks of work per domain—and produces artifacts that are difficult to reproduce and costly to maintain. This tension between the growing need for domain ontologies and the cost of producing them manually has opened an active line of research in ontological automation.

In this context, large language models (LLMs) have emerged as natural candidates to reduce this cost: they generate structured text in standard formats, understand specialized technical vocabulary, and can condition their output through prompting strategies, context retrieval, and multi-agent architectures. Nevertheless, the literature lacks systematic and reproducible evaluations that measure their actual capabilities on real biomedical databases, with metrics that simultaneously cover syntactic correctness, normative structural quality, and functional fidelity to the domain.

This work designs and implements a reproducible experimental set to evaluate four generation strategies—direct prompt (E1), controlled vocabulary (E2), semantic RAG (E3), and the OntoGenix multi-agent pipeline (E4)—applied cross-referenced to three LLM models (gpt-4o, Llama 3.1 8B, and Qwen 2.5 Coder 7B) on four databases of the human cis-regulatory domain (FANTOM5, dbSUPER, HACER, and DiseaseEnhancer). The evaluation combines syntactic validation, OQuARE metrics, and proficiency questions.

The results show that the multi-agent pipeline produces the structurally richest ontologies but receives intermediate normative scores due to over-structuring; the semantic RAG maximizes relational richness with large models and requires retrieval calibration in compact models. Performance is strongly model-dependent: no strategy exhibits stable ranking among the three LLMs evaluated.

Keywords

Biomedical ontologies, large language models, automatic ontology generation, RAG semantics, ontological engineering, OntoGenix, cis-regulatory domain, competency questions.

Índice de Contenido

Resumen	2
Palabras Clave	2
Abstract	3
Keywords	3
Índice de Contenido	4
Índice de Tablas	6
Índice de Figuras	7
1. Introducción	8
1.1. Objetivos	9
1.2. Estructura de la memoria	10
2. Estado del arte y marco teórico	10
2.1. Ontologías biomédicas y OWL 2	10
2.2. Generación de ontologías con modelos de lenguaje grandes	11
2.3. Evaluación automática de la calidad ontológica	13
2.4. Reproducibilidad y crisis de la replicación	14
3. Materiales y Métodos	15
3.1. Conjuntos de datos de validación	16
3.2. Tipo y tamaño de muestreo	20
3.3. Ontologías cisreg de referencia	21
3.4. Modelos LLM utilizados	21
3.5. Técnicas de contexto	22
3.6. Post-procesado mecánico	25
3.7. Sistema de evaluación	26
3.8. Diseño experimental	29
4. Resultados	33
4.1. Determinación del muestreo: sensibilidad conjunta al tipo y al tamaño	33
4.2. Calibración del sistema RAG (experimento E3)	35
4.3. Banco principal: triple validación con los parámetros fijados	37
5. Discusión	44
5.1. Reproducibilidad implementada	44
5.2. Discusión por pregunta de investigación	44
5.3. Lecciones metodológicas transferibles	45

5.4. Limitaciones	45
5.5. Líneas de trabajo futuro.....	45
6. Conclusiones	46
Referencias bibliográficas	47
Anexo A. Descripción de columnas de las bases de datos.....	50
Anexo B. Abreviaturas y siglas	52
Anexo C. Tests estadísticos: descripción, justificación y limitaciones	53
C.1. Bootstrap pareado de la diferencia de medias.....	53
C.2. Wilcoxon de rangos con signo (pareado).....	53
C.3. McNemar (proporciones pareadas).....	53
C.4. Tamaño de efecto (Cohen's d).....	54
C.5. Análisis de potencia y mínima diferencia detectable	54
C.6. Corrección por comparaciones múltiples (Bonferroni-Holm).....	55
C.7. Síntesis de limitaciones.....	55
Anexo D. Plantillas de prompts utilizadas.....	55
D.1. System prompt común (E1–E3).....	55
D.2. Indicación E1 — Zero-shot.....	55
D.3. Indicación E2 — Vocabulario controlado	56
D.4. Indicación E3 — RAG semántico real (annotationRAG).....	56
D.5. Indicación E4 — OntoGenix.....	56
Anexo E. Corpus de preguntas de competencia	57
Anexo F. Vocabulario controlado utilizado en E2 ({vocab_terms})	58
F.1. Lista literal de términos (22 entradas)	58
F.2. Notas sobre la materialización de la lista.....	59

Índice de Tablas

Tabla 1 Corpus de datos del dominio cis-regulatorio utilizado.....	17
Tabla 2 Diferencias entre las cuatro BD en tres ejes	19
Tabla 3 Tipos de muestreo evaluados en el banco transversal.	20
Tabla 5 Parámetros de generación comunes a los 4 modelos.....	22
Tabla 6 Pipeline replicada de OntoGenix paso a paso	25
Tabla 9 Métricas estructurales OQuaRE empleadas (once métricas en cinco sub-características).....	28
Tabla 10 Plan de réplicas del banco experimental.	32
Tabla 11 Sensibilidad conjunta al tipo de muestreo y al tamaño, sobre gpt-4o.....	34
Tabla 12 Calibración del RAG en E3	36
Tabla 13 Parse_ok antes/después del post-procesado, por estrategia.	37
Tabla 14 Errores sintácticos tipados por modelo (salida cruda, banco C_stratified·100).....	38
Tabla 15 OQuaRE Global medio (válidas/total) por estrategia y modelo	38
Tabla 16 Media de cada subcaracterística OQuaRE por modelo (réplicas válidas de E1–E4).....	39
Tabla 17 Densidad estructural por modelo y estrategia.....	40
Tabla 18 Fidelidad léxica y cobertura CQ, por estrategia y modelo.	41
Tabla 19 Síntesis cross-model: validez, OQuaRE, canon y cobertura CQ medias por modelo.....	42
Tabla 20. E4 (OntoGenix) por modelo, banco C_stratified·100.	42
Tabla 21 Jaccard medio inter-BD por estrategia y modelo.....	43
Tabla 22 Mínima diferencia detectable	54

Índice de Figuras

Figura 1 Proceso general de generación y evaluación de ontologías.	16
Figura 2 Diagrama del post-procesado mecánico.....	26
Figura 3 Pipeline experimental	29
Figura 4 Sensibilidad al tamaño muestral N	35
Figura 5 Calibración del sistema RAG	37
Figura 6 OQuaRE Global por experimento y modelo.....	39
Figura 7 Características OQuaRE por modelo	39
Figura 8 Riqueza estructural n_triples medio	40
Figura 9 Fidelidad al vocabulario cis regulatorio	41
Figura 10 Alucinaciones terminológicas por estrategia y modelo	43

1. Introducción

Una ontología, en el sentido computacional del término, es un artefacto de representación del conocimiento que especifica de forma explícita y formal los conceptos de un dominio, sus propiedades, las relaciones que los vinculan y las restricciones lógicas que los gobiernan (Studer et al., 1998). En bioinformática, las ontologías se expresan habitualmente en OWL 2 (*Web Ontology Language 2*), un estándar del W3C basado en lógicas de descripción que habilita la inferencia automática de conocimiento implícito y la consulta declarativa mediante SPARQL (Cuenca-Grau et al., 2008).

La ausencia de representaciones ontológicas actualizadas no es una limitación meramente técnica, sino que tiene consecuencias directas sobre la investigación biomédica. Sin ontologías formales, la integración de bases de datos genéticas heterogéneas requiere transformaciones ad hoc costosas y difícilmente reproducibles; el razonamiento automático sobre relaciones reguladoras —necesario, por ejemplo, para inferir qué enhancers son relevantes en un tipo celular dado— resulta imposible; y la consulta estandarizada sobre el conjunto del dominio queda fuera del alcance de equipos sin recursos de ingeniería del conocimiento (Mihindukulasooriya et al., 2023). La pregunta práctica que subyace a este problema es concreta: si un equipo biomédico dispone de un tiempo y recursos limitados para anotar semánticamente una fuente de datos genética nueva, ¿qué combinación de herramientas y técnicas le ofrece el mejor punto de partida con el menor esfuerzo humano?

La llegada de los modelos de lenguaje grandes (del inglés *Large Language Models*, LLM) ha abierto una nueva vía para abordar este problema. Los LLMs son sistemas de aprendizaje automático basados en la arquitectura Transformer (Vaswani et al., 2017), entrenados sobre corpus masivos de texto mediante objetivos de predicción de la siguiente unidad léxica a escala de cientos de miles de millones de parámetros. Este entrenamiento a escala les confiere una capacidad notable para generar texto estructurado en formatos arbitrarios, seguir instrucciones complejas y adaptar su salida al contexto proporcionado en un prompt (Brown et al., 2020). Aplicada al dominio de la ingeniería del conocimiento, esta capacidad generativa permite producir ontologías serializadas en lenguajes formales como Turtle o Manchester Syntax con un rendimiento que varía considerablemente en función de la estrategia de prompting empleada (Babaei Giglou, D'Souza y Auer, 2023).

La literatura especializada identifica cuatro familias principales de estrategias para la generación de ontologías con LLMs. La generación *zero-shot* consiste en solicitar al modelo la ontología directamente a partir de una descripción textual del dominio, sin ejemplos adicionales (Mateiu y Groza, 2023; Babaei Giglou, D'Souza y Auer, 2023). El prompting con vocabulario controlado restringe la salida del modelo a un esquema predefinido y a identificadores de ontologías existentes, obteniendo representaciones más coherentes con estándares establecidos (Caufield et al., 2024). La generación aumentada por recuperación (*Retrieval-Augmented Generation*, RAG) inyecta en el prompt conocimiento externo recuperado dinámicamente de una base de conocimiento, mejorando la fidelidad léxica y la cobertura del dominio (Lewis et al., 2020; Nayyeri et al., 2025). Finalmente, los pipelines multi-agente descomponen la tarea en etapas especializadas coordinadas por múltiples agentes; la aproximación más sofisticada de esta familia, *OntoGenix* (Val-Calvo et al., 2025), emplea tres agentes secuenciales y reporta ontologías estructuralmente más ricas que el prompting directo (Fathallah et al., 2024; Buehler, 2024).

Pese a estos avances, la revisión de la literatura —desarrollada en detalle en el capítulo 2— revela limitaciones metodológicas relevantes: la evaluación se realiza casi exclusivamente sobre modelos

propietarios de acceso cerrado (Meyer et al., 2023); no existe un banco comparativo unificado que enfrente las cuatro familias de estrategias sobre los mismos datos y las mismas métricas (Mihindukulasooriya et al., 2023); y la implementación del componente RAG se limita habitualmente a mecanismos de recuperación léxica básicos, sin explotar el RAG semántico basado en *embeddings* densos (Reimers y Gurevych, 2019). Estos vacíos definen el espacio en el que se inscribe el presente trabajo.

1.1. Objetivos

Este Trabajo de Fin de Máster propone un marco experimental reproducible para la generación automática de ontologías biomédicas en el dominio cis-regulatorio, evaluado sobre bases de datos reales y modelos de lenguaje de naturaleza complementaria. El objetivo general del trabajo es doble: establecer dicho marco e implementar las cuatro estrategias de generación, y evaluar comparativamente su rendimiento a lo largo de los tres ejes que definen el espacio operativo —el modelo, la estrategia de inyección de contexto y la estructura del dato de entrada—, para proporcionar una guía operativa a equipos biomédicos con recursos limitados. A continuación se detallan los objetivos generales y específicos que articulan este propósito.

1.1.1. Objetivos generales

OG1. Establecer un marco reproducible para la generación automática y el análisis comparativo de ontologías biomédicas mediante modelos de lenguaje grandes, evaluado sobre bases de datos del dominio cis-regulatorio.

OG2. Evaluar la replicabilidad de dicho marco a lo largo de los tres ejes que constituyen su espacio operativo —el modelo, la estrategia de inyección de contexto y la estructura del dato de entrada— cuantificando qué dimensión domina el resultado y proporcionando una guía operativa para futuros despliegues.

1.1.2. Objetivos específicos

OE1. Diseñar y ejecutar un banco experimental que combine las principales familias de estrategias de generación —prompt engineering, generación aumentada por recuperación (RAG) y pipelines multi-agente— sobre distintos modelos de lenguaje y distintas bases de datos del dominio cis-regulatorio.

OE2. Replicar funcionalmente una pipeline multi-agente de referencia sobre el corpus de bases de datos, con soporte añadido para modelos open-source ejecutables localmente.

OE3. Implementar un sistema de RAG semántico real basado en *embeddings* densos e índice vectorial sobre un corpus de ontologías biomédicas, y calibrarlo para distintos tamaños de modelo.

OE4. Establecer un sistema de evaluación a tres niveles complementarios —validación sintáctica, validación estructural mediante métricas normativas con razonamiento lógico, y validación funcional combinando fidelidad léxica al dominio y preguntas de competencia—.

OE5. Desarrollar un módulo de post-procesado mecánico determinista que separe el problema sintáctico del problema semántico y cuantificar su efecto.

OE6. Caracterizar los patrones de error sistemáticos de cada modelo y cada interacción modelo × estrategia mediante estadística cuantitativa y análisis cualitativo.

OE7. Documentar el banco con un marco de reproducibilidad alineado con la literatura de la crisis de la replicación: manifiestos de integridad de las entradas, versionado de los modelos y niveles de garantía explícitos.

1.2. Estructura de la memoria

La memoria se organiza en seis capítulos principales y cuatro anexos.

El presente capítulo 1 introduce el contexto del problema, justifica la relevancia de abordarlo, describe brevemente los enfoques existentes y sus limitaciones, y formula el objetivo general y los objetivos específicos del trabajo. El capítulo 2 desarrolla el estado del arte y el marco teórico sobre los cuatro ejes en los que se inscribe el trabajo: representación formal en OWL 2, generación de ontologías con LLMs, sistemas RAG semánticos y evaluación normativa de la calidad ontológica.

El capítulo 3, Materiales y Métodos describe el banco experimental: los conjuntos de datos, las ontologías de referencia, los modelos empleados y las cuatro estrategias de generación. Incluye las plantillas de prompts utilizadas, el sistema de evaluación a tres niveles —validación sintáctica, estructural y funcional—, el módulo de post-procesado mecánico y un apartado de diseño experimental que sintetiza la combinatoria completa de réplicas planificadas y las réplicas por celda.

El capítulo 4, Resultados reporta los hallazgos del banco. Se inicia con el análisis de sensibilidad al muestreo (diseño factorial conjunto $\text{tipo} \times \text{tamaño}$), que determina la pareja óptima $C_{\text{stratified}} \cdot N=100$ para el resto del banco. A continuación presenta los resultados de la triple validación, la calibración del sistema RAG, la comparativa cross-model que extiende cada estrategia a los tres modelos, y análisis transversales de consistencia inter-bases de datos.

El capítulo Discusión integra los resultados con las preguntas de investigación, discute la reproducibilidad implementada, las limitaciones metodológicas y las lecciones transferibles.

El capítulo 6 Conclusiones resume los logros principales, la contribución científica y técnica del trabajo, y las líneas de continuación. La memoria cierra con las referencias bibliográficas y cuatro anexos: descripción de columnas de las bases de datos, abreviaturas y siglas, reporte completo de tests estadísticos, plantillas literales de prompts y corpus de preguntas de competencia.

2. Estado del arte y marco teórico

Este capítulo proporciona el marco teórico y revisa la literatura reciente sobre los cuatro ejes en los que se inscribe el presente trabajo: la representación formal de ontologías OWL 2, la generación automática de ontologías mediante modelos de lenguaje grandes y la evaluación normativa de la calidad ontológica. El capítulo cierra con una discusión del problema general de la reproducibilidad en investigación con LLMs, marco que justifica las decisiones metodológicas adoptadas en el capítulo 3.

2.1. Ontologías biomédicas y OWL 2

Una ontología, en el sentido de Studer, Benjamins y Fensel (1998), es una especificación formal y explícita de una conceptualización compartida. En la web semántica se expresan habitualmente en

OWL 2 (Motik, Patel-Schneider y Parsia, 2012), serializadas en Turtle (W3C, 2014), con una sintaxis orientada a triples sujeto–predicado–objeto. Los elementos fundamentales son las clases (owl:Class), las propiedades de objeto (owl:ObjectProperty), las propiedades de datos (owl:DatatypeProperty), las propiedades de anotación (owl:AnnotationProperty —típicamente rdfs:label, rdfs:comment, skos:prefLabel, skos:definition—), las restricciones (owl:Restriction) y los axiomas subsumptivos (rdfs:subClassOf). Las propiedades de anotación, aunque carentes de semántica formal para el razonador, son fundamentales para la documentación y para el alineamiento con vocabularios canónicos del dominio.

Las ontologías biomédicas representan jerarquías de entidades biológicas, propiedades genómicas y relaciones funcionales. Las más establecidas (Gene Ontology, ChEBI, OBI, SNOMED-CT) cuentan con miles de clases curadas manualmente. Las ontologías más jóvenes, como las del dominio cis-regulatorio (CRM, enhancer-gene regulation), aún están en desarrollo activo y carecen de la cobertura de las primeras. Construir las manualmente presenta varios desafíos específicos: requiere la convergencia de competencias en semántica formal, lógica de descripción y conocimiento biomédico del dominio; los esquemas resultantes son frágiles ante la actualización del conocimiento biológico subyacente; y la propia diversidad de bases de datos genéticas, cada una con su esquema tabular y su nomenclatura, dificulta la integración. Esto abre espacio para sistemas automáticos o semi-automáticos que asistan al experto humano en la fase de modelado inicial.

Para tareas de razonamiento sobre ontologías ya construidas, OWL 2 define tres perfiles computacionalmente tratables —Lenguaje de Ontología Existencial (EL, por sus siglas en inglés), Lenguaje de Reglas (RL, por sus siglas en inglés) y Lenguaje de Consulta (QL, por sus siglas en inglés)— (Cuenca-Grau et al., 2008), junto con un perfil de Lógica de Descripción (DL, por sus siglas en inglés) más expresivo que mantiene la completitud y decidibilidad sobre la lógica SROIQ, utilizado por razonadores como HermiT (Glimm et al., 2014). Estos perfiles, sin embargo, son más relevantes para la fase de inferencia que para la de generación; en este TFM solo se utilizan en la validación estructural (sección 3.5.2) para verificar la consistencia DL de las ontologías generadas.

2.2. Generación de ontologías con modelos de lenguaje grandes

La aplicación de modelos de lenguaje grandes a tareas de ingeniería ontológica ha experimentado un crecimiento sostenido desde 2022. Cuatro paradigmas dominan la literatura, y este Trabajo de Fin de Máster los agrupa en tres categorías metodológicas que estructuran tanto la revisión del estado del arte como la evaluación experimental del capítulo 4. Cada categoría se describe a continuación en términos de su funcionamiento, los trabajos representativos publicados, y los desafíos o vacíos específicos que esa familia de estrategias presenta y que motivan el diseño del presente trabajo.

2.2.1. Prompt Engineering

La primera categoría agrupa las estrategias en las que el contexto necesario para guiar al modelo se inyecta directamente en el cuerpo del prompt, sin ningún mecanismo de recuperación dinámica. Dos variantes principales son la generación zero-shot, donde el modelo recibe únicamente una descripción del dominio o un esquema tabular y debe producir la ontología sin ejemplos previos, y el vocabulario controlado, donde se le proporciona una lista cerrada de términos ontológicos canónicos y se le pide que ajuste la salida con ese vocabulario mediante skos:exactMatch o skos:closeMatch. Una variante

intermedia es el few-shot, en el que se inyectan uno o varios pares (input, ontología) como ejemplo estructural antes de la consulta real.

Trabajos actuales. Meyer et al. (2023) reportan que GPT-4 produce ontologías válidas en zero-shot para esquemas simples, pero con alta variabilidad entre ejecuciones. Brown et al. (2020) formalizan el patrón few-shot como técnica general para inducir comportamientos específicos en LLMs. Mateiu y Groza (2023) evalúan vocabulario controlado sobre tres dominios distintos y demuestran que reduce la dispersión léxica frente al zero-shot, a costa de limitar la riqueza relacional. Neuhaus (2023) ofrece una perspectiva crítica sobre las limitaciones del prompt engineering como paradigma único para la construcción ontológica.

Gaps y desafíos. La principal limitación del prompt engineering puro es que no aporta conocimiento de dominio que no estuviera ya en los pesos del modelo: el LLM debe inferir la estructura ontológica de cero o reaplicar lo aprendido en el pre-entrenamiento. Esto produce inconsistencia entre ejecuciones, propensión a errores sintácticos en Turtle y, en modelos abiertos compactos, fallos sistemáticos no recuperables sin guía explícita. Adicionalmente, la literatura reciente carece de comparaciones controladas entre zero-shot y vocabulario controlado sobre el mismo banco con varios modelos, lo que impide cuantificar el beneficio incremental real del segundo. Este TFM aborda ese vacío evaluando ambas variantes bajo idénticas condiciones de modelo, semilla y datos (capítulo 4).

2.2.2. RAG Systems

La segunda categoría engloba las estrategias en las que el contexto se construye dinámicamente mediante un sistema de Retrieval-Augmented Generation: dada una consulta (el CSV del que se quiere generar la ontología), se recuperan fragmentos relevantes de un corpus ontológico previamente indexado y se inyectan en el prompt antes de invocar al modelo. El sistema descansa sobre tres componentes: un modelo de embeddings que mapea cada fragmento ontológico a un vector denso, un índice vectorial que soporta búsqueda aproximada por similitud coseno, y una capa de orquestación que construye el contexto inyectado al LLM con los k fragmentos más cercanos a la consulta. La primera generación de sistemas utilizaba retrieval léxico (BM25, TF-IDF, solapamiento de palabras clave), mientras que la generación actual emplea embeddings densos generados con sentence-transformers indexados en bases de datos vectoriales como Pinecone, Weaviate o Neo4j con índice vectorial nativo. Los parámetros principales son la dimensión del embedding, la granularidad del chunk indexado (axioma, clase, módulo o esquema completo), el valor de k y el umbral de score de corte; su elección determina qué fragmentos entran al contexto y, en consecuencia, la calidad de la generación.

Trabajos actuales. Lewis et al. (2020) formalizan el patrón RAG para tareas intensivas en conocimiento. Norouzi et al. (2024) aplican RAG a ontology alignment con ChatGPT. Reimers y Gurevych (2019) introducen sentence-transformers, base del retrieval semántico moderno; entre sus modelos, all-MiniLM-L6-v2 (Wang et al., 2020) es la opción por defecto en muchos sistemas por su tamaño compacto (22 MB, 384 dimensiones) y latencia muy baja, aunque no está entrenado específicamente para terminología biomédica. En el dominio biomédico, embeddings especializados como SapBERT (Liu et al., 2021), BioBERT (Lee et al., 2020) y BiomedBERT (Gu et al., 2021) reportan mejor recuperación que los de propósito general sobre terminología clínica, a costa de mayor huella computacional; la elección entre embeddings genéricos y especializados implica un compromiso entre fidelidad de dominio, coste y disponibilidad de hardware. Caufield et al. (2024) introducen SPIRES, un sistema de extracción zero-shot que populariza el patrón RAG aplicado a ontology learning. El presente trabajo emplea annotationRAG, un microservicio FastAPI desarrollado por el grupo Tecnomod (Universidad de

Murcia) que combina sentence-transformers con índice vectorial Neo4j 5.26 sobre 21 ontologías del dominio cis-regulatorio y vocabularios auxiliares.

Gaps y desafíos. El término «RAG» se utiliza con frecuencia para designar mecanismos de recuperación elementales que distan del retrieval vectorial real: muchos trabajos publicados emplean keyword-match estático sobre un conjunto pequeño de esquemas, lo que limita la generalización del hallazgo. Adicionalmente, los parámetros del retrieval (top_k, score_threshold, granularidad del chunk) suelen reportarse sin justificación empírica, y la interacción entre la riqueza del contexto recuperado y la capacidad de absorción del modelo subyacente está prácticamente sin documentar: ¿qué configuración funciona para un modelo cerrado de gran capacidad? ¿se traslada a un modelo abierto compacto? Este TFM ataca ese vacío implementando un RAG semántico real con índice vectorial Neo4j sobre 21 ontologías biomédicas y reportando un experimento de calibración del retrieval sobre dos modelos de capacidades muy distintas (capítulo 4.5).

2.2.3. Multi-agent

La tercera categoría corresponde a pipelines en las que la tarea de generación se descompone en sub-tareas asignadas a agentes LLM especializados que se invocan secuencialmente. Un patrón típico es: un agente planifica la estructura ontológica a partir del input, un segundo agente construye el TBox (clases, propiedades, restricciones), y un tercer agente mapea las instancias del CSV al ABox final. La descomposición permite especializar el prompt de cada agente y aplicar validaciones intermedias, a costa de multiplicar las invocaciones al modelo y, por tanto, el coste computacional y económico.

Trabajos actuales. OntoGenix (Val-Calvo et al., 2025) es un referente actual de la generación de ontologías mediante pipelines multi-agente —propuesto como método de propósito general, no aplicado al dominio cis-regulatorio— y emplea tres agentes (LlmPlanner, LlmOntology, LlmOntoMapper) sobre OpenAI GPT-4. OntoGPT (Caufield et al., 2024) integra grounding contra ontologías de referencia para reducir la generación libre de URIs ad-hoc. KGGen (Kommineni et al., 2024) extiende el patrón multi-agente a la construcción de grafos de conocimiento generales más allá del dominio biomédico.

Gaps y desafíos. La complejidad orquestal de las pipelines multi-agente incrementa la riqueza estructural de la salida pero introduce dos problemas. Primero, la dependencia de un modelo cerrado específico: OntoGenix está diseñado para GPT-4 y su replicación con modelos abiertos no está documentada, lo que limita su adopción en entornos con restricciones de privacidad o coste. Segundo, el coste computacional puede ser prohibitivo cuando se aplica a grandes volúmenes de datos. Tercero, no existe una comparación controlada entre el método multi-agente y las baselines de prompt engineering sobre el mismo banco que cuantifique qué parte de la mejora reportada es atribuible al método y qué parte al modelo subyacente. Este TFM responde al primer y al tercer vacío replicando funcionalmente OntoGenix con tres adaptaciones técnicas documentadas que habilitan su ejecución headless y con modelos open-source vía Ollama (capítulo 3.4.3).

2.3. Evaluación automática de la calidad ontológica

La evaluación automática de ontologías generadas combina tres dimensiones complementarias, no reducibles unas a otras. La validación sintáctica comprueba que la serialización parsea y que la ontología es consistente bajo un razonador estándar (HermiT, Pellet, ELK); es condición necesaria para cualquier evaluación posterior, pues una ontología que no parsea no puede medirse. La evaluación

estructural cuantifica propiedades del grafo subyacente sin recurrir al significado del dominio; las tres familias más citadas son cohesión (interconectividad interna), modularidad (independencia entre módulos) y complejidad (densidad axiomática), pero la literatura ha propuesto un catálogo mucho más amplio: acoplamiento, número de clases raíz y hoja, profundidad y anchura de la jerarquía, tangledness, densidad relacional y las cuatro medidas de riqueza de Tartir et al. (2005) —riqueza de atributos, de relaciones, de clases y de herencia (Yao et al., 2005; Vrandečić y Sure, 2007). La evaluación semántica intenta capturar la adecuación al dominio; los dos indicadores más habituales son la fidelidad léxica (solapamiento con vocabulario canónico) y la riqueza axiomática (densidad de axiomas SubClassOf, disjointness y restricciones), pero también se emplean la cobertura conceptual sobre un corpus de dominio (Brank et al., 2005), la corrección de las relaciones inferidas mediante validación experta muestrada, la información contenida (information content) y la satisfacción de preguntas de competencia formuladas en SPARQL (Grüniger y Fox, 1995).

Trabajos actuales. OQuaRE (Duque-Ramos et al., 2014) es el marco normativo más establecido y ejemplifica una integración parcial de las tres dimensiones: adapta ISO/IEC 25000 —SQuaRE, originalmente para software— al dominio ontológico, define varias sub-características (structural, reliability, operability, maintainability, compatibility, transferability, functional adequacy) y las operacionaliza mediante once métricas con rúbricas publicadas en escala 1–5, produciendo un score global. Existen esfuerzos posteriores por cubrir las tres dimensiones de forma explícita: S-OntoEval (Dividino et al., 2008) propone niveles estructural, funcional y pragmático basados en teoría semiótica, y el marco SynEvaluator/SemValidator (Amith et al., 2022) combina reglas sintácticas configurables con validación semántica basada en expertise externo. En el ámbito biomédico, NCBO Recommender (Martínez-Romero et al., 2017) y BioPortal Quality Score (Ochs et al., 2017) actúan como recomendadores no normativos enfocados a la selección y evaluación de ontologías publicadas.

Gaps y desafíos. Ninguno de los marcos disponibles integra las tres dimensiones con una métrica global objetiva y consensuada: OQuaRE agrega numéricamente varias sub-características pero la casi totalidad de sus métricas son estructurales, y no captura la corrección biomédica de las relaciones inferidas; S-OntoEval permanece a nivel teórico sin una implementación de referencia mantenida; SynEvaluator/SemValidator depende de validadores humanos externos, lo que limita su escalabilidad y reproducibilidad. La validación semántica sigue apoyándose en última instancia en el juicio experto (Bandeira et al., 2016), lo que la hace costosa y difícil de reproducir. Este TFM ataca dos vacíos: introduce métricas de fidelidad léxica automatizadas basadas en el gold set cisreg (apartado 4.3.3) y complementa OQuaRE con validación funcional mediante preguntas de competencia SPARQL (apartado 3.6.3), reportando las tres dimensiones de forma independiente para preservar la trazabilidad de cada lectura y evitar el sesgo de compresión en un score único.

2.4. Reproducibilidad y crisis de la replicación

La reproducibilidad —capacidad de un tercero de obtener los mismos resultados a partir del código, datos y protocolo publicados— es requisito básico del método científico. Gundersen y Kjensmo (2018) formalizaron una jerarquía de tres niveles según las garantías que ofrece el procedimiento reportado: R1 (reproducibilidad estricta, mismos números bit-a-bit), R2 (estadística, mismas conclusiones dentro del intervalo de confianza) y R3 (metodológica, mismas conclusiones bajo un protocolo equivalente pero no idéntico). El nivel alcanzable depende del control que el investigador tenga sobre las fuentes

de variabilidad de su banco: código, semillas, versión exacta del modelo, hardware, y —en el caso de sistemas basados en LLM— la política de sampling del proveedor.

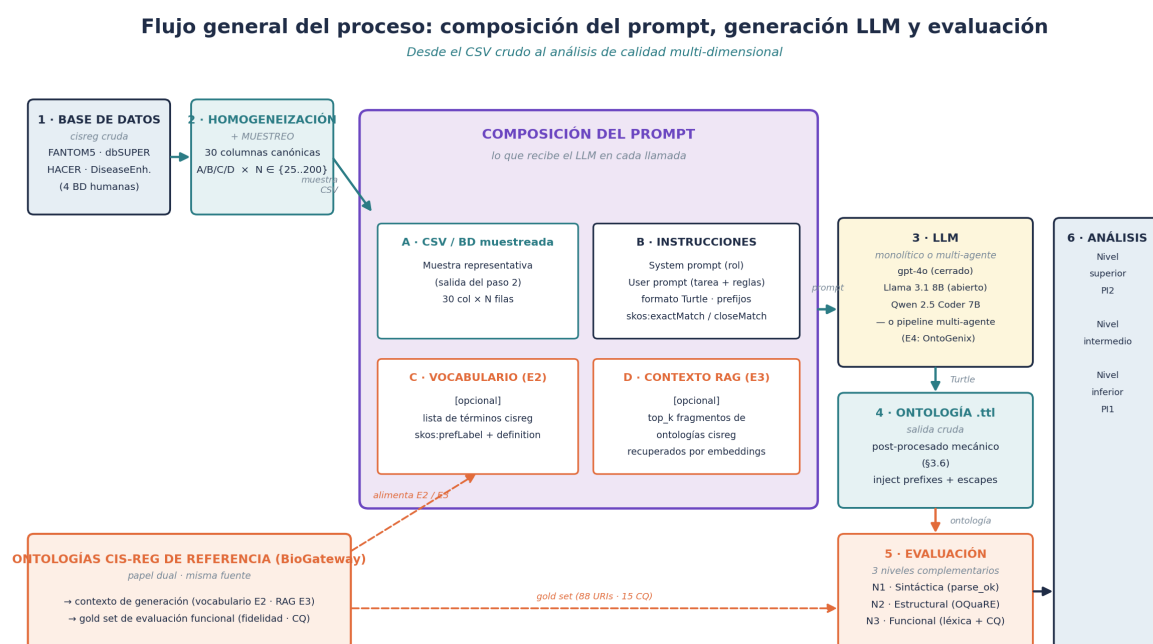
Trabajos actuales. La denominada «crisis de la replicación» fue identificada inicialmente en psicología (Open Science Collaboration, 2015) y en investigación preclínica (Begley y Ellis, 2012), y se ha extendido a la inteligencia artificial: Hutson (2018) estimó que entre el 70 % y el 90 % de los experimentos publicados en las grandes conferencias de IA no son reproducibles con la información proporcionada. Pineau et al. (2021) reportaron los resultados del programa de reproducibilidad de NeurIPS 2019 y propusieron un checklist que ha sido adoptado por las principales conferencias del área. En el ámbito específico de ontology engineering con LLMs, la práctica de reportar prompt literal, versión de modelo, seed y parámetros de generación no está consolidada, y la proporción de trabajos que publican código, datos y ontologías generadas junto con la memoria sigue siendo minoritaria.

Gaps y desafíos. La investigación con LLMs introduce amenazas específicas a la reproducibilidad. Primero, el determinismo bit-a-bit no se garantiza ni con temperature=0 y seed fija, en parte por la concurrencia interna del proveedor (Henderson et al., 2018), lo que sitúa R1 fuera del alcance de los modelos cerrados vía API. Segundo, los alias de modelo (p. ej. gpt-4o) son móviles y apuntan a snapshots distintos en función del momento de la llamada, lo que impide reproducir el modelo exacto una vez el proveedor lo actualiza. Tercero, las cuantizaciones de modelos open-source (Q4_K_M, Q5_K_S) pueden cambiar entre re-descargas si el mantenedor re-empaqueta el modelo bajo la misma etiqueta. Y cuarto, los embeddings publicados se actualizan sin notificación de versionado, alterando silenciosamente los resultados de cualquier RAG que dependa de ellos. Este TFM encuadra estas amenazas en la taxonomía R1/R2/R3 (apartado 5.1), explicita las limitaciones de reproducibilidad de cada componente del banco, fija snapshots concretos de modelo y embedding, y libera código, datos, prompts y ontologías generadas para permitir replicación al menos al nivel R2.

3. Materiales y Métodos

Este capítulo describe el banco experimental, las herramientas y los datos empleados para la evaluación comparativa de las cinco estrategias de generación. La descripción sigue el orden del pipeline mostrado en la Figura 1: conjuntos de datos de validación y su homogeneización a un esquema columnar canónico (3.1), tipo y tamaño de muestreo (3.2), ontologías cisreg de referencia como recurso transversal a la generación y a la evaluación (3.3), modelos LLM utilizados (3.4), técnicas de contexto agrupadas en tres categorías metodológicas con sus prompts (3.5), post-procesado mecánico (3.6), sistema de evaluación a tres niveles (3.7) y diseño experimental que sintetiza la combinatoria completa (3.8). La Figura 1 ofrece una visión global del proceso antes de detallar cada etapa en los apartados siguientes.

Figura 1 Proceso general de generación y evaluación de ontologías.



El diagrama de la figura 1 se lee de arriba a abajo y de izquierda a derecha en tres franjas. En la franja superior izquierda están los pasos de preparación de los datos (cajas 1 y 2) que fijan el CSV muestreado. En la franja central, el rectángulo morado explicita los cuatro componentes que forman cada llamada al LLM —A la muestra CSV, B las instrucciones, C el vocabulario cisreg cuando la estrategia es E2, y D el contexto RAG cuando la estrategia es E3—; esas cuatro sub-cajas son la traducción explícita de «técnicas de contexto» del apartado 3.5 al nivel del prompt real. En la franja derecha e inferior, la ontología generada por el LLM (caja 4) atraviesa el post-procesado y las tres capas de evaluación (caja 5), cuyo resultado alimenta el análisis por niveles del capítulo 4 (caja 6). Las flechas naranjas discontinuas hacen visible el papel dual de las ontologías cis-reg de referencia: fuentes del contexto de generación y fuentes del gold set de evaluación funcional.

3.1. Conjuntos de datos de validación

El banco experimental utiliza cuatro bases de datos del dominio cis-regulatorio humano. Las cuatro comparten un mismo esquema columnar (treinta columnas canónicas obtenidas tras un pre-procesado de homogeneización) que cubre: identificador del módulo regulador, coordenadas genómicas en dos ensamblajes (hg19 y hg38), score de confianza, referencia PubMed, biosample, método experimental, tipo de regulación, fuente, genes diana, factores de transcripción, enfermedades asociadas y mutaciones registradas. Las características distintivas de cada una se resumen en la Tabla 1. Las diferencias en contenido entre bases de datos se muestran en la Tabla 2 (calculada sobre la muestra C_stratified de 100 filas fijada en el banco principal, §4.1).

Tabla 1 Corpus de datos del dominio cis-regulatorio utilizado.

Base de datos	Referencia	Cobertura	Entidades clave
FANTOM5	Andersson et al. (2014)	1.829 muestras de tejidos y líneas celulares humanas; ~37 000 enhancers activos.	TSS · CAGE peaks · cell types
dbSUPER	Khan y Zhang (2016)	102 tejidos/tipos celulares humanos; super-enhancers anotados manualmente.	Super-enhancer · TF · tissue
HACER	Wang et al. (2019)	1.408 muestras; asocia enhancers activos con sus genes diana por eQTL y co-expresión.	Active enhancer · target gene
DiseaseEnhancer	Zhang et al. (2018)	Enhancers asociados a enfermedades humanas mediante variantes funcionales y evidencias.	Enhancer · disease · evidence

La homogeneización a treinta columnas canónicas es una decisión metodológica deliberada, vamos a explicar brevemente en qué consiste:

El proceso opera sobre tres capas:

- Uniformiza los nombres de columna (por ejemplo, «chrom» de dbSUPER, la ausencia de cabecera en el BED de FANTOM5 y el «cell_name» de dbSUPER se mapean al canónico «orig_chr», «current_chr» y «biosample_name», respectivamente).
- Uniformiza los formatos de valor: convención «chr1» para el cromosoma, coordenadas enteras en ensamblajes hg19 y hg38 en columnas separadas (orig_ vs current_), guion «-» como marcador único de valor no disponible, codificación UTF-8 y separador TSV.
- Uniformiza el esquema del fichero: cada BD se proyecta sobre exactamente el mismo conjunto de treinta columnas, rellenando con «-» las que no aporta y descartando las que quedan fuera del esquema.

Tras el proceso, las cuatro BD del banco comparten literalmente las treinta columnas, lo que difiere es el grado de relleno (cuántas filas tienen valor real vs «-»), y ese es precisamente el eje que la Tabla 2 (§3.1.1) cuantifica.

¿Por qué treinta columnas?. El esquema canónico es la unión de los campos modelados por las cuatro ontologías cis-reg de referencia del grupo BioGateway (crm.ttl, crm2gene.ttl, crm2phen.ttl, crm2tfac.ttl), agrupadas en siete familias semánticas: identificador (crm_ID), coordenadas genómicas (orig_/current_chr/start/end/assembly), calidad de la evidencia (minimum_ratio, score), procedencia (original_ID, crossref, enh_PMIID, enh_method, source, type), contexto biológico (biosample_name), regulación gen→CRM (hgnc_symbol_target_genes, enh2gene_PMIID/method), unión con factores de transcripción (hgnc_symbol_TFs, TFs2enh_PMIID/method), asociación con enfermedades (disease, disease_PMIID, disease_method) y mutaciones registradas (refsnp_ID, mutation_PMIID, mutation_method).

Con este esquema se cubre el modelo de conocimiento cis-regulatorio de referencia y se hace explícita, para cada BD, qué aspectos aporta y cuáles deja vacíos, evitando que la varianza inter-BD se confunda con diferencias triviales de formato. El significado completo de cada una de las treinta columnas canónicas se documenta en el Anexo A.

Además del contenido tabular, cada muestra se antepone al prompt precedida por cinco líneas de metadatos con formato de comentario Turtle/Python (prefijo #) que identifican y contextualizan la base de datos. El formato de estas cinco líneas es idéntico entre bases de datos, pero su contenido cambia; a modo de ejemplo, para dbSUPER se envía:

```
# Base de datos: dbSUPER
# Descripción: Super-enhancers con coordenadas, líneas celulares y genes diana
# Fuente: https://asntech.org/dbsuper/
# Dimensiones totales: "dbSUPER": {
    "db_name": "dbSUPER",
    "file_size_mb": 15.78,
    "n_rows": 69022,
    "n_cols": 30,
    "analysis_sample_size": 10000,
    "completeness": {
        "avg_completeness_pct": 69.8,
        "min_completeness_pct": 0.0,
        "max_completeness_pct": 100.0,
        "gene_coverage_pct": 98.1,
        "disease_coverage_pct": 0.0,
        "tf_coverage_pct": 0.0
    }
}
# Número de columnas: 30
```

Estas cinco líneas contienen tres campos que varían libremente entre bases de datos —el nombre, la descripción y la fuente— y dos campos que quedan fijos por el pre-procesado de homogeneización (dimensiones y número de columnas). Los tres primeros son, por tanto, una fuente adicional de variabilidad inter-BD que el diseño no controla explícitamente: el modelo puede orientar la generación al reconocer términos como «Super-enhancer», «disease» o «CAGE-seq» en la línea de descripción antes de inspeccionar la muestra propiamente dicha, lo que se traduciría en un sesgo léxico de la ontología resultante. Este efecto se discute en el capítulo 5 al interpretar las diferencias residuales entre BD que no se explican por el contenido tabular.

3.1.1. Diferencias informativas entre las cuatro bases de datos

Pese a compartir el esquema columnar homogeneizado, las cuatro BD aportan información sustantivamente distinta. La Tabla 2 sintetiza las diferencias en tres ejes: las filas de metadatos del fichero original, los nombres y formatos de los conceptos principales antes de la homogeneización, y la cobertura semántica del dominio que cada BD aporta. Estas diferencias son relevantes porque la pregunta de investigación PI1 (apartado 1.2) plantea si el resultado del banco depende del modelo, la técnica de contexto y la estructura de los datos de entrada; este apartado caracteriza esa última dimensión.

Las cuatro BD comparten un mismo esquema canónico de 30 columnas tras la homogeneización y todas usan la convención «chr1» en las columnas de cromosoma (`orig\_chr` y `current\_chr`). Las diferencias relevantes para el banco no están en el formato sino en qué columnas vienen rellenas y en qué medida, lo que determina la riqueza semántica máxima que el modelo puede inducir desde cada BD. La Tabla 2 reporta la tasa de relleno (filas con datos faltantes y distinto de «-») para las columnas más informativas sobre las muestras de 25 filas que se inyectan al modelo en el banco principal.

Tabla 2 Diferencias entre las cuatro BD en tres ejes

Columna	FANTOM5	dbSUPER	HACER	DiseaseEnh.
Formato del fichero raw	TSV (BED-like)	TSV con cabecera	TSV con cabecera	TSV con cabecera
Nº columnas del fichero raw	12	7	s.d.	s.d.
Nº filas totales (raw)	65 422	69 205	≈ 1 408 †	s.d.
Ensamblajes cubiertos	hg19 + hg38	hg19 + hg38	hg19 + hg38	hg19 + hg38
Convención de cromosoma	«chr1»	«chr1»	«chr1»	«chr1»
Nombre pre-hom. de «biosample_name»	biosample *	cell_name	biosample_name	biosample_name
Nombre pre-hom. de «refsnp_ID»	refsnp_ID	refsnp_ID	refsnp_ID	refseq_ID **
biosample / biosample_name	0/25 *	25/25	25/25	0/25
type (tipo de regulación)	0/25	25/25	0/25	0/25
enh_method (método experim.)	25/25	25/25	23/25	0/25
hgnc_symbol_target_genes	20/25	24/25	14/25	24/25
enh2gene_PMIID	20/25	24/25	14/25	24/25
hgnc_symbol_TFs	0/25	0/25	0/25	0/25
disease	0/25	0/25	0/25	25/25
disease_PMIID	0/25	0/25	0/25	25/25
refsnp_ID / refseq_ID **	0/25	0/25	2/25	10/25
mutation_PMIID	0/25	0/25	2/25	10/25
original_ID (xref externa)	0/25	25/25	2/25	25/25

Tres observaciones de la Tabla 2 son relevantes para la discusión del capítulo 5. Primera, la información sobre factores de transcripción (`hgnc\_symbol\_TFs`, junto con sus columnas de trazabilidad `TFs2enh\_PMIID` y `TFs2enh\_method`) está vacía en las cuatro BD del banco y en los cuatro tamaños de muestra evaluados ($N \in \{25, 50, 100, 200\}$ con muestreo estratificado $C_{\text{stratified}}$: 0/25, 0/50, 0/100, 0/200 filas con valor real en las cuatro BD). Ninguna induce, por tanto, relaciones CRM $\leftrightarrow$ TF en la ontología generada. Aun así, las tres columnas se mantienen en el esquema canónico por alineación estructural con la ontología cis-reg de referencia: *crm2tfac.ttl* modela justamente el eje CRM $\leftrightarrow$ TF, y su vocabulario forma parte del gold set contra el que se calcula la fidelidad léxica (§3.7.3).

Suprimir estas columnas rompería la simetría datos $\leftrightarrow$ ontología de referencia y penalizaría al modelo si generara clases como TranscriptionFactor o propiedades como bindsTo —que sí existen en el gold set— alegando falta de soporte en el input. Adicionalmente, el esquema fijo permite acomodar sin renegociación futuras BD que sí cubran este eje (ReMap, UniProt), cuya integración se propone como línea de trabajo futuro en el capítulo 6. Segunda, la información sobre enfermedades sólo está presente en DiseaseEnhancer, y la información sobre tipo de regulación («super-enhancer») sólo está presente en dbSUPER; el resto de aspectos (cobertura experimental, genes diana, PMIIDs) está repartido de forma desigual. Tercera, esta heterogeneidad en la cobertura columnar es una variable

explicativa del rendimiento dispar de las técnicas de contexto: el RAG semántico, por ejemplo, recupera fragmentos del corpus indexado que cubren mejor unos aportes que otros, lo que se discute en el apartado 5.2.1.

3.2. Tipo y tamaño de muestreo

Las cuatro BD del corpus contienen entre 1 408 y 69 205 filas. Por razones de presupuesto de tokens y latencia, el banco no inyecta la BD completa al modelo, sino una muestra representativa. Tanto el tipo de muestreo (cómo se seleccionan las filas) como el tamaño de la muestra (cuántas filas se inyectan) son variables del diseño experimental cuya influencia sobre los resultados se estudia explícitamente en el capítulo 4. Por su carácter de variable independiente, este apartado se separa del apartado 3.1 y describe los cuatro tipos de muestreo evaluados y los cuatro tamaños del barrido.

3.2.1. Tipos de muestreo

Manteniendo constantes las cinco líneas de metadatos del prompt (§3.1), el apartado se centra en la selección del bloque de datos en formato TSV que las acompaña. Las cuatro estrategias evaluadas para esa selección se describen en la Tabla 3.

Tabla 3 Tipos de muestreo evaluados en el banco transversal.

Etiqueta	Tipo	Descripción	Cobertura categórica
A_head	Cabecera	Primeras filas del fichero ordenado por la fuente.	Baja · sesgo de orden
B_random	Aleatorio	Selección uniforme con semilla fija (42).	Alta · uniforme
C_stratified	Estratificado	Una pasada categórica sobre la columna más informativa.	Muy alta · proporcional
D_diversity	Diversidad	Clustering k-means sobre embeddings; centroide por cluster.	Máxima · representatividad

3.2.2. Tamaño de la muestra

El tamaño de la muestra —parámetro N — es el número de filas del bloque de datos TSV que sigue a las cinco líneas de metadatos del prompt (§3.1). Ese bloque se compone, a su vez, de dos sub-bloques deterministas: cinco filas de cabecera tomadas del inicio de la base de datos (parámetro $n_header=5$ en `scripts/02_sample_databases.py`, invariante en todas las configuraciones para garantizar que el modelo vea siempre el mismo ejemplo canónico) y $N-5$ filas adicionales seleccionadas según la estrategia de muestreo (§3.2.1). El barrido factorial del apartado 4.1 explora tres tamaños adicionales, $N \in \{50, 100, 200\}$ —esto es, $5 + 45$, $5 + 95$ y $5 + 195$ filas muestreadas— de forma conjunta con el tipo de muestreo.

El barrido $N \in \{25, 50, 100, 200\}$ cuantifica la influencia del tamaño de la muestra sobre los resultados y permite responder a la dimensión «tamaño de muestra» de la pregunta de investigación PI1. Sus resultados se reportan en el apartado 4.1, en el diseño factorial conjunto `tipoxtamaño`.

3.3. Ontologías cisreg de referencia

Las ontologías de referencia del dominio cis-regulatorio proceden del proyecto BioGateway (Universidad de Murcia) y modelan ocho aspectos complementarios: `crm.ttl` (esquema básico de módulos cis-reguladores), `crm_example.ttl` (instanciación con valores concretos), `crm2gene.ttl` (relaciones CRM $\leftrightarrow$ genes diana) y su versión de ejemplo, `crm2phen.ttl` (relaciones CRM $\leftrightarrow$ fenotipos) y su versión de ejemplo, y `crm2tfac.ttl` (relaciones CRM $\leftrightarrow$ factores de transcripción) y su versión de ejemplo. Los esquemas declaran URIs canónicas bajo el espacio de nombres `http://rdf.biogateway.eu/`, que se reutilizan en las ontologías generadas mediante alineamiento explícito (`skos:exactMatch` / `skos:closeMatch`). Nota sobre el uso de ambas anotaciones en este TFM: aunque `skos:exactMatch` (equivalencia fuerte, transitiva) y `skos:closeMatch` (similitud débil, no transitiva) tienen semánticas distintas en la especificación SKOS, en el cómputo de las métricas de fidelidad léxica de §3.7.3 ambas se tratan como equivalentes —cualquier alineamiento SKOS a una URI del namespace canónico cuenta como reuso de vocabulario, con independencia de la fuerza de la relación—. Es una limitación consciente del diseño: analizar por separado el uso de cada propiedad exigiría una métrica adicional que verificase la adecuación semántica del alineamiento (equivalencia real vs similitud aproximada) contra el gold set, lo que queda propuesto como trabajo futuro (§5.5). Estas ocho ontologías constituyen también el gold set utilizado en la métrica de fidelidad léxica (apartado 4.3.3), del que se extraen 88 URIs canónicas únicas como referencia. Estas ontologías pueden consultarse en tres puntos: la descripción normativa de los grafos `crm/gene/prot` en la página oficial de BioGateway (<https://biogateway.eu/graphs/>), el endpoint SPARQL público del proyecto (<https://biogateway.eu/sparql-endpoint/>) —que permite dereferenciar cualquiera de las URIs del namespace canónico `http://rdf.biogateway.eu/`— y una copia local versionada de los ocho ficheros Turtle en el propio repositorio del TFM bajo `data/samples/schemas/{crm,crm2gene,crm2phen,crm2tfac}.txt` (esquema) y sus versiones `..._example.txt` (instanciación), reutilizadas por el sistema RAG (§3.5.2).

3.4. Modelos LLM utilizados

La evaluación se realiza sobre tres modelos de capacidades muy distintas, seleccionados para cubrir el rango operativo realista de un equipo biomédico y responder a la pregunta de investigación PI2 (¿existe una estrategia estable entre modelos?). La terna combina, en un mismo banco, el techo comercial (`gpt-4o`), un modelo abierto generalista desplegable en hardware de consumo (Llama 3.1 8B) y un modelo abierto especializado en código de tamaño equivalente (Qwen 2.5 Coder 7B), con lo que se contrastan simultáneamente el eje cerrado $\leftrightarrow$ abierto, el eje generalista $\leftrightarrow$ especialista y —dentro de los abiertos— dos arquitecturas ampliamente adoptadas por la comunidad. Los modelos de más de ~8B parámetros (Llama 3.1 70B, Qwen 72B) se dejan fuera del banco principal por coste de inferencia local y se reservan como línea de trabajo futuro (§5.5).

- `gpt-4o` (snapshot 2024-05-13; OpenAI). Modelo cerrado de propósito general, accesible vía API. Representa la frontera de capacidad disponible comercialmente (OpenAI, 2024).
- Llama 3.1 8B Instruct (Meta). Modelo abierto generalista, accesible localmente vía Ollama con cuantización Q4_K_M. Representa la baseline open-source de tamaño compacto desplegable en hardware de consumo (Meta AI, 2024; Ollama, 2024).

- Qwen 2.5 Coder 7B (Alibaba). Modelo abierto especializado en generación de código, accesible vía Ollama. Permite contrastar si la especialización en código se traduce en mejor sintaxis Turtle (Hui et al., 2024; Ollama, 2024).

Los tres modelos se invocan a través de una API compatible con OpenAI: OpenAI directamente para gpt-4o, y Ollama (puerto local 11434) con wrapper compatible para los dos modelos abiertos (Ollama, 2024). Los parámetros de generación son comunes (temperature, top_p, seed) y se detallan en la Tabla 4.

3.5. Técnicas de contexto

Las cuatro estrategias evaluadas comparten un mismo input —una muestra del CSV original en formato TSV, precedida por cinco líneas de comentario con metadatos— y una misma salida esperada —un archivo Turtle válido que represente la ontología generada—. La diferencia entre estrategias reside en cómo se inyecta el contexto y en cómo se orquestan las llamadas al LLM. Las cuatro estrategias se agrupan en tres categorías metodológicas que estructuran tanto la descripción de este apartado como el análisis de resultados del capítulo 4.

Tabla 4 Parámetros de generación comunes a los 4 modelos.

Parámetro	Valor	Razón
temperature	0	Obliga al modelo a elegir siempre la palabra más probable.
top_p	1.0	Sin truncamiento adicional cuando temperature=0 (en la API de OpenAI 1.0 es el valor neutro).
top_k	1	Selecciona únicamente la palabra de máxima probabilidad. Cuando lo admite la API, sobrescribe a top_p.
seed	42	Semilla única que fija el inicio del generador para asegurar replicabilidad.
frequency_penalty	0	Sin penalización por repetición; preserva la lógica matemática pura del modelo.
presence_penalty	0	No fuerza al modelo a introducir temas nuevos de forma artificial.
max_tokens	8 192	Suficiente para una ontología extensa; 8 192 por agente en E4.
timeout	120 s / 300 s (E4)	120 s para E1–E3; 300 s por agente en E4.
formato de salida	Turtle plano	Sin marcadores Markdown ni explicaciones.

Con la configuración determinista de la Tabla 4, dos invocaciones idénticas a la API deberían producir la misma ontología. En la práctica, los proveedores no garantizan determinismo bit-a-bit por concurrencia interna del backend (Henderson et al., 2018). Para cuantificar empíricamente esta varianza, el banco principal ejecuta tres réplicas por cada experimento con la misma configuración determinista; las diferencias entre las tres réplicas miden el nivel real de reproducibilidad del proveedor y se reportan junto con la media en todas las tablas del capítulo 4.

3.5.1. Categoría I — Prompt Engineering

Esta categoría agrupa las estrategias en las que el contexto se inyecta directamente en el cuerpo del prompt del modelo, sin recuperación dinámica. El system prompt común a todas las estrategias E1–E3 establece el rol del modelo como ontólogo experto en grafos biomédicos. El user prompt incluye el nombre de la base de datos, su descripción funcional, la muestra (100 filas, banco principal) y un bloque variable de contexto. La muestra se conserva como texto TSV plano para preservar la estructura tabular original.

E1 — Generación zero-shot

El modelo recibe la muestra del CSV y la instrucción genérica de producir una ontología en Turtle siguiendo el patrón estándar (prefijos rdf, rdfs, owl, skos, xsd, obo). No se proporciona ejemplo, vocabulario ni esquema de referencia. La estrategia mide el rendimiento basal del LLM sobre la tarea. La plantilla literal del prompt se reproduce en el Anexo D.

E2 — Vocabulario controlado

El prompt incluye, además del CSV, una lista compacta de vocabulario controlado extraída automáticamente de los esquemas cisreg (líneas skos:prefLabel y skos:definition), generada en tiempo de ejecución por la función load_vocabulary. El procedimiento aplica un tope máximo de ochenta términos por compatibilidad con la ventana de contexto, pero en el banco ejecutado ese tope nunca se alcanzó: los cuatro esquemas incluidos aportan sólo veintidós entradas únicas tras deduplicación, que se reproducen literalmente en el Anexo F. El modelo recibe la instrucción de preferir esos términos al nombrar clases y propiedades, y de declarar alineamientos mediante skos:exactMatch / skos:closeMatch cuando corresponda. Esta variante puede entenderse como una forma de few-shot indirecto donde el ejemplo es léxico, no estructural. Para permitir la auditoría del vocabulario sin ejecutar la función, la salida literal de load_vocabulary() sobre los cuatro esquemas actuales queda persistida en [data/samples/vocab_terms_E2.txt](#) del repositorio del TFM.

Selección del prompt óptimo

Antes de fijar las plantillas literales del Anexo D, se realizaron varias iteraciones de ingeniería de prompts sobre la categoría I (zero-shot y vocabulario controlado) con gpt-4o sobre dbSUPER. Se compararon cuatro variantes que modificaban: (i) el orden de las instrucciones (lista de pasos antes vs. después de la muestra del CSV), (ii) la inclusión o no de una directiva explícita de cadena de razonamiento («think step by step»), (iii) la lista de prefijos canónicos a usar, y (iv) la instrucción de salida (Turtle plano vs. explicación + Turtle). La variante final, fijada y empleada en todo el banco principal, es la que: incluye la lista de pasos antes del CSV, mantiene la directiva de razonamiento paso a paso, exige los prefijos rdf, rdfs, owl, skos, xsd y obo, y obliga a producir únicamente Turtle plano sin marcadores Markdown. Esta combinación elevó la tasa de parse-OK nativo de gpt-4o-mini del 0 % al 83 % en dbSUPER, antes de aplicar el post-procesado mecánico del apartado 3.6. La estrategia que mejor funciona, por tanto, es el prompt iterado de la categoría I; los detalles de todas las variantes intermedias se guardan como ficheros dedicados en el directorio [data/prompts/](#) del repositorio del TFM, con un README que documenta los cuatro ejes evaluados y un fichero por variante (E1_final.prompt, E1_variant_A_lista_despues.prompt, E1_variant_B_sin_cot.prompt y E1_variant_C_con_explicacion.prompt) más el system_prompt.txt común. La fuente autoritativa en producción sigue siendo [scripts/run_gpt_experiments.py](#) (constantes SYSTEM_PROMPT y

USER_PROMPT_E1/E2), cuyo historial git y las versiones .bak de [scripts/](#) [archivo/](#) completan la trazabilidad.

3.5.2. Categoría II — RAG Systems

La segunda categoría engloba las estrategias en las que el contexto se construye dinámicamente mediante un sistema de recuperación basado en embeddings densos sobre un corpus ontológico indexado. Este TFM implementa una única estrategia bajo esta categoría: E3, basada en el microservicio `annotationRAG` con índice vectorial Neo4j sobre 21 ontologías del dominio cis-regulatorio y vocabularios auxiliares. La motivación para esta elección es doble: por un lado, el retrieval vectorial es el estado del arte actual del RAG aplicado a ingeniería ontológica; por otro, el espacio de diseño del RAG semántico es lo suficientemente amplio (parámetros `top_k`, `score_thr`, `max_chars`, modelo de embeddings) como para concentrar en él un análisis empírico riguroso (apartado 4.5). La línea de continuidad natural —combinar el retrieval semántico con un mecanismo de recuperación léxica para explotar la consistencia inter-BD del segundo— se reserva como trabajo futuro (capítulo 6).

E3 — RAG semántico real (*annotationRAG*)

Implementación basada en el microservicio [annotationRAG](#) (Riquelme-García et al., 2025). Los componentes son:

- Splitter de chunks: cada ontología se descompone en chunks correspondientes a clases individuales (axioma owl:Class y sus restricciones asociadas).
- Modelo de embeddings: [sentence-transformers/all-MiniLM-L6-v2](#) (384 dimensiones; Wang et al., 2020).
- Base de datos vectorial: [Neo4j 5.26 con índice vectorial nativo](#) y métrica de similitud coseno.
- Corpus indexado: 21 ontologías (8 cisreg de referencia procedentes de [BioGateway](#) y 13 auxiliares de dominios complementarios).
- Recuperación: el cliente HTTP construye una consulta a partir del CSV, llama a los endpoints `/similar-ontologies` y `/similar-entities`, y filtra por `top_k` y umbral de score; el contexto inyectado consiste en metadata estructurada (clases, propiedades, scores) más fragmentos Turtle de las ontologías más similares.

Los parámetros principales del recuperador (`top_k=5`, `score_thr=0.4`, `max_chars=5000` en la configuración baseline) son objeto de calibración específica en el apartado 4.5.2 para evaluar su efecto sobre modelos compactos.

Artefactos de reproducibilidad en el repositorio del TFM: la guía operativa completa para levantar el servicio (docker-compose, corpus e indexado inicial) está en [docs/SETUP_RAG.md](#); el manifiesto Docker Compose de Neo4j 5.26 con el índice vectorial se encuentra en [docker-compose.neo4j.yml](#); el cliente HTTP que consume `/similar-ontologies` y `/similar-entities` es [scripts/rag_backend.py](#); y la reimplementación funcional del servidor `annotationRAG` —compatible en contrato HTTP y arquitectura pero con pesos locales— está en [scripts/ragannotation_server.py](#). Esta reimplementación reproduce el contrato y la arquitectura del servicio original del grupo Tecnomod, no los pesos exactos, lo que sitúa E3 en el nivel R3 (metodológico) de la taxonomía de reproducibilidad (§2.4).

3.5.3. Categoría III — Multi-agent

La tercera categoría comprende las pipelines en las que la tarea de generación se descompone en sub-tareas asignadas a agentes especializados que se invocan secuencialmente. El TFM evalúa el referente actual del dominio: OntoGenix.

E4 — Pipeline multi-agente OntoGenix

OntoGenix (Val-Calvo et al., 2025) descompone la generación en tres agentes especializados que se invocan secuencialmente (Tabla 5). La pipeline original requiere acceso a SERP API para enriquecer el contexto con resultados de búsqueda web. En la replicación funcional realizada para este TFM se omite el paso SERP (no se dispone de credencial) y se documentan tres adaptaciones técnicas mínimas: bypass del módulo Searcher, bypass del paso SERP y parche en LlmBase para soportar Ollama como proveedor alternativo a OpenAI. Las plantillas literales de cada agente están disponibles en el directorio [GUI/](#) del repositorio OntoGenix (tecnomod-um).

Tabla 5 Pipeline replicada de OntoGenix paso a paso

Paso	Agente	Función	Entrada	Salida
1	LlmPlanner	Analiza el CSV. Detecta entidades, relaciones, jerarquía.	CSV completo + metadata	Plan en JSON (clases, propiedades, jerarquía)
2	LlmOntology	Construye el TBox: clases, propiedades, restricciones, anotaciones.	Plan + CSV	Turtle inicial (TBox)
3	LlmOntoMapper	Mapea cada fila a instancias de las clases declaradas.	TBox + CSV completo	Turtle final con ABox

3.6. Post-procesado mecánico

Para separar el problema sintáctico (sin valor científico) del semántico (objeto de evaluación), se desarrolló un módulo de post-procesado determinista que aplica tres tipos de transformaciones en cascada:

- Auto-prefix injection, que detecta prefijos referenciados pero no declarados (linkml, dc, obo, biolink, etc.) y los añade desde un diccionario canónico.
- Escape de local names problemáticos, que reescribe como IRI absoluta cualquier token pname con caracteres no válidos en PN_LOCAL.
- Saneado de literales mal tipados, que sustituye literales con valor textual tipado como entero por xsd:string. El módulo es completamente determinista y no invoca al LLM, lo que permite separar limpiamente la dimensión sintáctica de la semántica. Su efecto se analiza estadísticamente con McNemar.

Orden en el proceso y condicionalidad. El post-procesado se sitúa entre la salida cruda del LLM y el sistema de evaluación de §3.7. Sobre cada .ttl generado se aplica en cascada, en este orden: (1) parseo con rdflib para computar parse_ok crudo (parse_before) y clasificar el fallo, cuando lo hay, en las tres categorías tipadas de §3.7.1; (2) pasada estándar (auto-prefix + escape de local names) aplicada de

forma incondicional a todos los ficheros E1–E3; (3) sólo si tras la pasada estándar el fichero sigue sin parsear, pasada agresiva de recuperación (saneado de literales, terminadores y términos bare) que se conserva únicamente si consigue el parseo —nunca puede empeorar un fichero ya válido; (4) parseo final para computar parse_after. E4 (OntoGenix) se evalúa sobre la salida cruda por decisión de diseño: la pipeline multi-agente aporta su propia validación intermedia y su comparación es de método frente a las baselines, no de post-procesabilidad. La implementación completa está en [scripts/postprocess_ttl.py](#).

Entrada al sistema de evaluación. La triple validación de §3.7 opera sobre la versión post-procesada del .ttl cuando existe, y sólo cae a la raw en E4 o en los casos residuales sin postprocessed/ generado. En el discovery de ficheros ([scripts/evaluate_three_levels.py](#)), cuando una ontología tiene contraparte en postprocessed/ se salta la raw y se evalúa la reparada. En consecuencia, la métrica sintáctica que entra en la comparación final del capítulo 4 es parse_after (para E1–E3) o parse_before (para E4, que no tiene post-procesado). Las dos cifras se reportan lado a lado en la Tabla 10 para hacer explícito el rescate del post-procesado y se contrastan con la prueba de McNemar; el resto de niveles (OQuaRE + razonador, fidelidad léxica, cobertura CQ) se calcula sobre el conjunto con parse_after = 1.

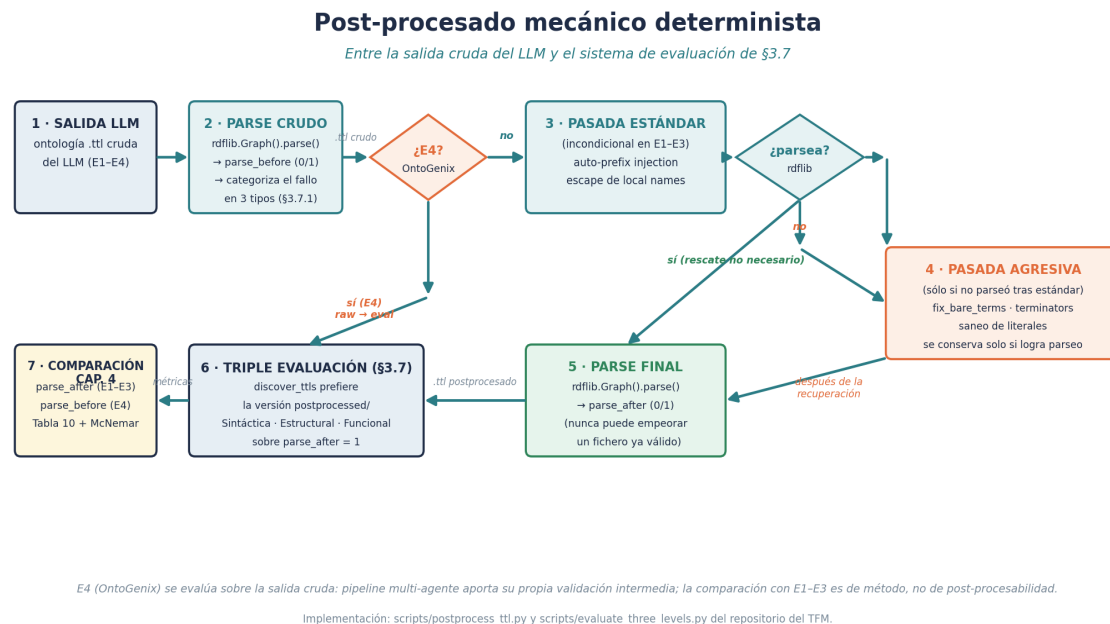


Figura 2 Diagrama del post-procesado mecánico.

Flujo entre la salida cruda del LLM y la triple evaluación de §3.7. La pasada estándar es incondicional en E1–E3; la pasada agresiva de recuperación sólo se activa si tras la estándar el fichero sigue sin parsear y se conserva únicamente si consigue el parseo. E4 evita el post-procesado y entra directamente a la evaluación con la salida cruda.

3.7. Sistema de evaluación

La evaluación se realiza en tres niveles complementarios. Cada nivel responde a una dimensión diferente de calidad y aporta una lectura propia al análisis de resultados. Reportar los tres es lo que permite, en el capítulo 4, discriminar entre estrategias que premian la estructura pero abandonan el

vocabulario establecido (E4) y estrategias que reusan vocabulario pero producen estructuras pobres (E3).

3.7.1. Validación sintáctica

La validación sintáctica se apoya en dos indicadores complementarios, no en uno solo. El primero es `parse_ok`, un binario 0/1 que vale 1 si `rdflib` carga la ontología sin excepción y 0 en caso contrario; es el filtro de entrada al banco, y las salidas con `parse_ok=0` se descartan de los análisis posteriores. El segundo, complementario y no reemplazo, es un recuento categorizado de errores sintácticos —distinto del `parse_ok`— que este TFM introduce para enriquecer la lectura más allá del binario y que se reporta en la Tabla 11 (§4.3.1). Este segundo indicador clasifica cada fallo de parseo en tres ejes complementarios:

- Errores de URIs/prefijos. Prefijos referenciados pero no declarados (`linkml`, `dc`, `obo`, `biolink`, etc.), URIs con namespaces ad-hoc no resolubles, y violaciones del esquema `pname` (caracteres no admitidos en `PN_LOCAL`: `!`, `#`, `/`, `@`).
- Errores de caracteres. Literales con caracteres no escapados, comillas no balanceadas, mezcla de delimitadores de serializaciones RDF distintas (`Turtle` + `RDF/XML` + `JSON-LD` en el mismo archivo) y omisión del «:» del prefijo en declaraciones del tipo «`crm` a `owl:Class`».
- Errores de etiquetas y literales. Literales con valor textual tipados erróneamente como numérico (`xsd:integer`, `xsd:decimal`, `xsd:boolean`), `rdfs:label` con caracteres no válidos, y conflictos entre el dato declarado y el tipo asignado.

3.7.2. Validación estructural — OQuaRE + razonador

La validación estructural combina dos componentes complementarios. El primero es la puntuación normativa OQuaRE (Duque-Ramos et al., 2014; framework abierto y sus rúbricas oficiales en el repositorio [tecmod-um/oquare-metrics](https://tecmod.um.es/oquare-metrics)), implementada en [scripts/oquare_eval.py](https://github.com/tecmod-um/oquare-metrics/blob/main/scripts/oquare_eval.py): se computan once métricas estructurales sobre el grafo RDF asertado —es decir, sobre los axiomas explícitamente insertados por el LLM, sin cierre por inferencia— usando `rdflib` y las clases/propiedades enumeradas por `owlready2`. Estas once métricas se agregan en cinco sub-características (Structural, Modularity, Reusability, Operability, Reliability) y producen un OQuaRE Global en escala 1–5. La Tabla 6 enumera las once métricas y su sub-característica.

El segundo componente es un check binario de consistencia OWL DL ejecutado por el razonador HermiT (Glimm et al., 2014) a través de `owlready2`, independiente del cómputo de las once métricas OQuaRE. Cada ontología generada se carga en `owlready2`, se invoca `sync_reasoner()` y se registra el resultado como un valor booleano: pasa el razonador (la ontología es consistente y no tiene clases insatisfacibles) o no pasa (HermiT detecta inconsistencias o clases insatisfacibles, lo que invalida la ontología desde el punto de vista lógico). El razonador no expande la ontología ni alimenta las métricas OQuaRE; opera como filtro lógico posterior. Este check binario es independiente de la puntuación OQuaRE: una ontología puede obtener una puntuación OQuaRE alta y no pasar el razonador si declara axiomas contradictorios, e inversamente una ontología puede pasar el razonador y obtener una puntuación OQuaRE baja por ser estructuralmente pobre. Reportar los dos componentes por separado en el capítulo 4 permite discriminar estos casos.

Tabla 6 Métricas estructurales OQuARE empleadas (once métricas en cinco sub-características).

Sub-característica	Métricas	Dirección
Structural	WMOnto, NOMOnto, DITM, NACOnto	Menor = mejor
Modularity	CBOnto, TMOnto, LCOMOnto	Menor = mejor
Reusability	ANOnto	Mayor = mejor
Operability	INROnto, RROnto, AROnto	Mayor = mejor
Reliability	HermiT consistency	Booleano

3.7.3. Validación funcional — fidelidad léxica + preguntas de competencia

Como aproximación al problema de la falta de ground truth biomédico, este trabajo introduce dos métricas complementarias que evalúan la utilidad operativa de las ontologías generadas.

Fidelidad léxico-semántica al dominio. Se construye un gold set a partir de las ocho ontologías cisreg de referencia —crm.ttl, crm_example.ttl, crm2gene.ttl, crm2gene_example.ttl, crm2phen.ttl, crm2phen_example.ttl, crm2tfac.ttl y crm2tfac_example.ttl—, del que se extraen 88 URIs canónicas únicas. La construcción del gold es determinista y reproducible: la función `build_gold_uris` de [scripts/cisreg_fidelity.py](#) recorre los ficheros anteriores —versionados en [data/samples/schemas/](#) del repositorio del TFM—, extrae por regex todas las URIs con formato `prefix:local` o `<IRI absoluta>`, y descarta las que caen en namespaces estándar no autoritativos del dominio (`rdf`, `rdfs`, `owl`, `xsd`) para dejar sólo las URIs con anclaje semántico en el dominio cis-regulatorio. Para cada ontología generada se computan cinco métricas complementarias:

- `n_canonical`: número de URIs de la ontología generada cuyo namespace pertenece al gold del dominio (`rdf.biogateway.eu`, `purl.obolibrary.org/obo/`, `w3id.org/biolink/`, `identifiers.org/`, `ebi.ac.uk`, `ncbi.nlm.nih.gov` y otros namespaces autoritativos biomédicos). Mide reuso de vocabulario establecido.
- `n_invented`: número de URIs declaradas por el LLM en namespaces claramente ad-hoc (`example.org/*`, `purl.example.org/*`, etc.). Mide invención espuria.
- `canonical_ratio` = $n_canonical / (n_canonical + n_invented)$: proporción entre 0 y 1 que resume el equilibrio entre reuso e invención. Un valor de 1,00 significa «todas las URIs de la ontología generada están en namespaces del dominio»; un valor de 0,00 significa «todas están inventadas».
- `overlap_with_gold` = $|gold \cap generated|$: número de las 88 URIs canónicas del gold que aparecen literalmente en la ontología generada. Es un contador absoluto entre 0 y 88 y mide reuso concreto (no sólo dominio correcto, sino URI exacta).
- `recall_vs_gold` = $overlap_with_gold / 88$: proporción del gold canónico que la ontología generada cubre. Complementa a `canonical_ratio`: `canonical_ratio` alta con `recall_vs_gold` baja indica una ontología que reutiliza vocabulario del dominio pero sólo un subconjunto muy limitado del gold canónico.

En conjunto, `canonical_ratio` y `recall_vs_gold` responden a dos preguntas distintas: «¿el modelo inventa o reutiliza vocabulario del dominio?» y «¿qué fracción del vocabulario canónico llega efectivamente a la ontología?». La métrica discrimina entre estrategias que reusan vocabulario establecido y estrategias que inventan URIs ad-hoc en namespaces `example.org/`.

Validación funcional mediante preguntas de competencia. Se construye un corpus reproducible de 15 preguntas de competencia (competency questions, CQ; Grüninger y Fox, 1995) del dominio cis-regulatorio, formuladas en SPARQL canónico y acompañadas de un patrón de requisitos TBox: el conjunto mínimo de clases y predicados que la ontología debe declarar para que la consulta sea ejecutable en principio. Las preguntas cubren cinco dimensiones: localización genómica, regulación gen $\leftrightarrow$ CRM, unión factor de transcripción, asociación a fenotipo / enfermedad, evidencia experimental y contexto biológico.

3.8. Diseño experimental

Cada experimento del banco se define unívocamente por la combinación de tres componentes que el resto de la memoria emplea de forma consistente para nombrar y analizar los resultados. La figura que sigue sintetiza esta estructura y detalla los valores concretos que toma cada componente en el banco principal.

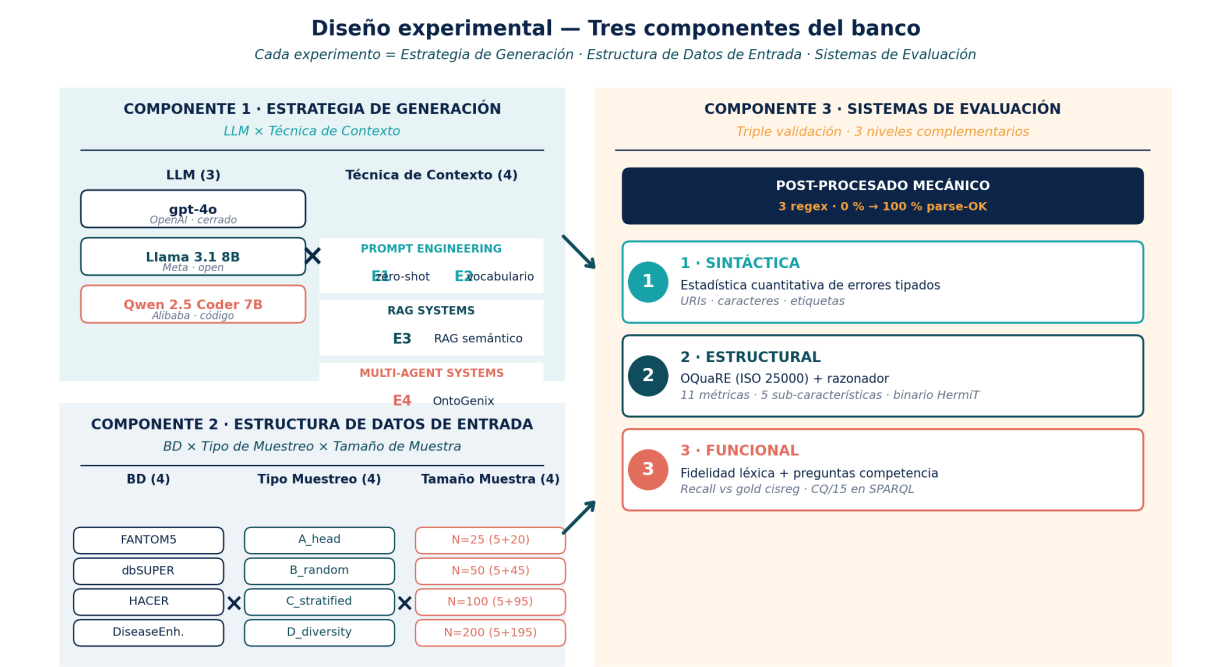


Figura 3 Pipeline experimental

Cada experimento es la combinación de una Estrategia de Generación (LLM × Técnica de Contexto), una Estructura de Datos de Entrada (BD × Tipo de Muestreo × Tamaño) y los Sistemas de Evaluación de tres niveles aplicados uniformemente sobre la ontología post-procesada.

3.8.1. Tres componentes de un experimento

Componente 1 — Estrategia de Generación. Producto cartesiano del conjunto de modelos de lenguaje subyacente y el conjunto de técnicas de inyección de contexto: Estrategia de Generación = LLM × Técnica de Contexto. Los modelos disponibles se describen en el apartado 3.4 (gpt-4o, Llama 3.1 8B Instruct, Qwen 2.5 Coder 7B). Las técnicas de contexto se describen en el apartado 3.5 y se agrupan en tres categorías: Prompt Engineering (E1 zero-shot, E2 vocabulario controlado), RAG Systems (E3 RAG semántico) y Multi-agent systems (E4 OntoGenix).

Componente 2 — Estructura de Datos de Entrada. Producto cartesiano de los tipos de base de datos, los tipos de muestreo y los tamaños posibles de la muestra: Estructura de Datos de Entrada = BD × Tipo de Muestreo × Tamaño de Muestra. Las cuatro BD se describen en el apartado 3.1 (FANTOM5, dbSUPER, HACER, DiseaseEnhancer). Los cuatro tipos de muestreo (A_head, B_random, C_stratified, D_diversity) y los cuatro tamaños (25, 50, 100, 200) se describen en el apartado 3.2.

Componente 3 — Sistemas de Evaluación. Tres niveles complementarios descritos en el apartado 3.7: validación sintáctica con estadística cuantitativa de errores en URIs, caracteres y etiquetas (3.7.1); validación estructural con la puntuación normativa OQuaRE y el check binario del razonador HermiT (3.7.2); validación funcional combinando fidelidad léxico-semántica al gold cisreg con un corpus de 15 preguntas de competencia en SPARQL (3.7.3).

Cada experimento se ejecuta con tres réplicas con la misma configuración determinista de la Tabla 4. Las tres réplicas permiten establecer la media de las métricas cuantitativas y cuantificar el nivel de reproducibilidad del proveedor, dado que las APIs comerciales no garantizan determinismo bit-a-bit pese a la configuración (Henderson et al., 2018).

3.8.2. Flujo experimental por fases

Sobre estos tres componentes, el banco combina cinco dimensiones factoriales —estrategia (4) × modelo (3) × BD (4) × tipo de muestreo (4) × tamaño N (4)—, más la configuración del RAG (4 valores en E3) y el prompt (con variantes discretas antes de su fijación).

Un barrido factorial completo sobre todas estas dimensiones exigiría del orden de $4 \times 3 \times 4 \times 4 \times 4 \times 4 \times 3 \approx 9\,200$ ejecuciones planificadas —más de un orden de magnitud sobre el banco actual, con un coste inasumible en tiempo de cómputo y en llamadas API de gpt-4o— sin garantía adicional de calidad.

En su lugar, el diseño adopta una estrategia secuencial en cuatro fases que resuelve las decisiones metodológicas de una en una, congela el resultado y avanza. El orden es deliberado: cada fase acota el espacio de configuración para la siguiente y evita la duplicación combinatoria; el precio es asumir separabilidad razonable entre las dimensiones fijadas y las que quedan por resolver, supuesto que se verifica empíricamente al comprobar en cada fase que el ranking OQuaRE es lo bastante plano como para no depender fuertemente de las interacciones con las dimensiones no barridas (Tabla 8 y Anexo C). Estas cuatro fases se resumen en la Tabla 7 con sus dimensiones libres, fijas y el número de corridas planificadas por fase.

Fase 0. Selección del prompt óptimo (§3.5.1). Sobre gpt-4o-mini y dbSUPER se comparan cuatro variantes de plantilla que difieren en:

- (i) El orden lista-de-pasos vs muestra CSV,
- (ii) Presencia de directiva «think step by step».
- (iii) Lista de prefijos canónicos.
- (iv) Formato de salida (Turtle plano vs explicación + Turtle).

La combinación ganadora eleva parse-OK nativo de gpt-4o-mini del 0 % al 83 % en dbSUPER y se congela para todo el banco. Justificación de la elección: se toma gpt-4o-mini como «canario» porque los problemas de format-following se manifiestan primero en modelos de menor capacidad —una plantilla que funciona con un modelo pequeño funciona mejor con uno grande—; dbSUPER se emplea porque presenta el esquema columnar de mayor cobertura relativa (§3.1.1) y por tanto exige más del

format-following del modelo. Esta fase no se ejecuta sobre las 3×4 combinaciones (modelo, BD): es una decisión de metodología, no de rendimiento.

Fase 1. Sensibilidad al muestreo (§4.1). Fija la pareja (tipo*, N*). Rejilla barrida: 4 tipos $\times$ 4 tamaños = 16 celdas por (modelo, BD, estrategia); ejecutada sobre 3 modelos $\times$ 4 BDs $\times$ 4 estrategias $\times$ 3 réplicas = 2 304 ejecuciones planificadas.

La decisión se toma con una regla lexicográfica fijada a priori (validez $\rightarrow$ OQuaRE Global $\rightarrow$ fidelidad léxica $\rightarrow$ cobertura CQ) evaluada sobre gpt-4o. La elección de gpt-4o como referente busca que el ranking dependa de la calidad estructural y funcional del banco, y no del ruido que introducirían réplicas con parse-OK fallido en los modelos abiertos.

Como salvaguarda contra el sesgo hacia gpt-4o, se reporta también la elección alternativa que produciría el agregado de los tres modelos y se contrasta con la mínima diferencia detectable con $n = 3$ réplicas (Anexo C).

Fase 2. Calibración del RAG (§4.2). Fija la configuración RAG*. Rejilla barrida: 4 configuraciones {baseline (top_k=5), C1 (top_k=2), C2 (score_thr=0,6), C3 (C1+C2)}. Ejecutada sobre 3 modelos $\times$ 4 BDs $\times$ 3 réplicas con (tipo*, N*) ya fijados y exclusivamente sobre E3, única estrategia que emplea RAG (144 ejecuciones planificadas). Calibrar sobre los tres modelos —y no sólo sobre gpt-4o— es deliberado, para detectar la dependencia del óptimo de retrieval con el tamaño del modelo. La decisión de RAG* se toma sobre gpt-4o siguiendo el mismo criterio de la fase 1; la dependencia con el tamaño del modelo se retoma en §4.5.2 y como línea de trabajo futuro en §5.5.

Fase 3. Banco principal (§4.3). Con prompt (fase 0), (tipo*, N*) (fase 1) y RAG* (fase 2) seleccionados, se ejecuta la comparativa transversal: 4 estrategias $\times$ 3 modelos $\times$ 4 BDs $\times$ 3 réplicas = 144 ejecuciones planificadas. Todas las tablas y figuras del capítulo 4 reportan, para cada agregación, tanto la media como la fracción $n_{\text{OK}} / n_{\text{total}}$ de réplicas válidas, para garantizar la comparabilidad entre celdas con distinto denominador.

El diseño secuencial fija cuatro decisiones (prompt, tipo de muestreo, tamaño N y configuración del RAG) tomándolas predominantemente sobre gpt-4o. Este es un sesgo asumido y declarado: representa el escenario de un usuario que dispone del modelo cerrado de referencia y calibra el banco para él. Para acotarlo, cada fase reporta cómo la decisión habría cambiado si el criterio se hubiese aplicado a los tres modelos agregados (fase 1) o se hubiese analizado por modelo (fase 2), y las diferencias observadas son consistentemente inferiores a la mínima diferencia detectable con $n=3$ réplicas. La comparativa cross-model del banco principal permanece, por tanto, informativa a pesar de la calibración gpt-4o-centric. Un banco factorial completo que evite este sesgo se plantea como línea de trabajo futuro (§5.5).

Tabla 7 Plan de réplicas del banco experimental.

Experimento	Dimensiones libres	Dimensiones fijas	Cálculo	Ejecuciones
Principal	LLM × Técnica × BD	C_stratified · N=100 · RAG*=baseline	3 × 4 × 4 × 3	144
Sensibilidad al muestreo	Tipo Muestreo × BD × LLM	Ninguna (factorial completo)	4×4 tipos×N × 3 mod × 4 est × 4 BD × 3 rép = 768	768
Calibración del RAG	Configuración RAG	Técnica E3 · C_stratified · N=100	3 mod × 4 conf × 4 BD × 3 rép = 144	36
Banco principal	LLM × Técnica × BD	C_stratified · N=100 · RAG*=baseline	4 est × 3 mod × 4 BD × 3 rép = 144	144

Las tablas y figuras del capítulo 4 reportan, para cada agregación, tanto la media como la fracción n_{OK} / n_{total} de réplicas que superaron la validación sintáctica, dado que una OQuaRE Global de 4,20 calculada sobre 12 réplicas válidas no es comparable con el mismo valor calculado sobre 6 réplicas. Esta práctica, alineada con la literatura de reproducibilidad (Pineau et al., 2021), se discute con detalle en el apartado 5.1.

3.8.3. Niveles de análisis de los resultados

Tras ejecutar los experimentos y pasar sus salidas por los tres sistemas de evaluación, los resultados se analizan en dos bloques, uno por tipo de experimento: los transversales (§4.1 y §4.2), que responden a una pregunta operativa cada uno y cuya salida es la fijación de un parámetro para el banco principal; y el experimento principal (§4.3), que se analiza en tres niveles complementarios estructurados por granularidad.

Experimentos transversales. Cada transversal responde a una única pregunta operativa cuya respuesta es un valor congelado para el banco principal, no una comparación multi-nivel. En §4.1 (sensibilidad al muestreo) la pregunta es «¿qué pareja (tipo*, N*) maximiza la calidad conjunta sobre el modelo de referencia gpt-4o, con validez casi total en toda la rejilla?», resuelta con una regla lexicográfica (validez → OQuaRE Global → fidelidad léxica → cobertura CQ) y complementada con una comprobación de robustez sobre el agregado de los tres modelos. En §4.2 (calibración del RAG) la pregunta es «¿qué configuración RAG* maximiza la calidad de E3 sobre gpt-4o?», resuelta con la misma regla; el análisis paralelo sobre los modelos compactos no cambia RAG* pero sí documenta la dependencia del óptimo con el tamaño del modelo (§4.5.2). En ambos casos, el resultado del transversal no se reinterpreta en el banco principal; sirve exclusivamente como parámetro fijado.

Experimento principal. Con las dimensiones ya congeladas por los transversales, los resultados del banco principal se analizan en tres niveles complementarios que estructuran la presentación del capítulo 4:

- Nivel superior. ¿Qué combinación de Estrategia de Generación y Estructura de Datos de Entrada da el mejor resultado en general, y por sistema de evaluación individual? Este nivel responde a la

pregunta de investigación PI2 (apartado 1.2) sobre la existencia de una combinación estable y se reporta en el apartado 4.3.4.

- Nivel intermedio. ¿Qué Estrategia de Generación, marginalmente sobre la Estructura de Datos, da el mejor resultado en general y por sistema de evaluación? ¿Y qué Estructura de Datos, marginalmente sobre la Estrategia, da el mejor resultado? Este nivel discrimina la contribución relativa de cada componente y se reporta en el apartado 4.3.
- Nivel inferior. Para cada sub-componente del banco principal (LLM, Técnica de Contexto y Base de Datos —las tres dimensiones libres tras congelar Tipo de Muestreo y Tamaño de Muestra en §4.1—), ¿cuál da el mejor resultado en general y por sistema de evaluación, y cómo varía respecto a los demás sub-componentes? Este nivel responde a la pregunta de investigación PI1 (apartado 1.2) sobre la dependencia del resultado respecto al modelo, la técnica y la estructura de los datos. Se reporta en el apartado 4.3.6 (análisis transversales dentro del banco principal, no confundir con los experimentos transversales §4.1–§4.2).

Esta doble estructura —transversales que resuelven decisiones puntuales, banco principal analizado en tres niveles— permite al lector situar cualquier afirmación cuantitativa del capítulo 4 en el componente y la granularidad correspondientes, y facilita la trazabilidad entre evidencia experimental, preguntas de investigación y conclusiones.

4. Resultados

Este capítulo sigue la secuencia lógica del banco: primero se fijan, con experimentos específicos, los tres parámetros de diseño que condicionan al resto —**tipo de muestreo**, **tamaño de muestra** y **configuración del RAG**—; después se ejecuta el banco principal completo con esos parámetros congelados. Así, cualquier comparación posterior por estrategia, modelo o base de datos es limpia y no mezcla configuraciones distintas.

Todos los experimentos, en todas las fases, se evalúan con el mismo sistema de tres niveles del apartado 3.7 (validación sintáctica, estructural y funcional); cada nivel mide una cualidad distinta y los tres se reportan siempre juntos. La Figura 1 resume el flujo del proceso.

Cálculo de los valores. Salvo indicación contraria, las medias se calculan sobre las réplicas válidas (`parse_ok` = 1) de cada celda. El indicador `parse_ok` es el binario 0/1 definido en §3.7.1: vale 1 si `rdflib` carga el fichero Turtle sin lanzar excepción y 0 en caso contrario. Las réplicas con `parse_ok` = 0 se descartan del cómputo de OQuaRE, fidelidad léxica y cobertura CQ porque no hay ontología parseable sobre la que medir; se contabilizan aparte en la fracción n_{OK} / n_{total} que acompaña cada media, de modo que el lector pueda distinguir una media alta calculada sobre pocas réplicas de una media alta robusta sobre todas.

4.1. Determinación del muestreo: sensibilidad conjunta al tipo y al tamaño

El banco inyecta una muestra de la base, no la base completa (apartado 3.2). Dos parámetros la definen: el **tipo de muestreo** (qué filas se eligen) y el **tamaño N** (cuántas). Para reducir sesgos se cruzan ambos factores en un único diseño factorial y se elige la pareja (tipo, N) de forma conjunta.

4.1.1. Diseño

Producto cartesiano de los cuatro tipos {A_head, B_random, C_stratified, D_diversity} por los cuatro tamaños $N \in \{25, 50, 100, 200\}$, con tres réplicas por celda, aplicado de forma completa a los tres modelos y a las cuatro estrategias (E1–E4) sobre las cuatro bases de datos. Única excepción: E4 con Llama 3.1 8B, que no produce ontologías válidas (0/12; apartado 4.3.5). La muestra de cada experimento se construye igual en todos los casos (cabecera de metadatos + N filas en TSV según el tipo). En total: 2 304 ejecuciones planificadas para esta fase (16 parejas (tipo × N) × 144 ejecuciones por pareja, donde $144 = 3 \text{ modelos} \times 4 \text{ BDs} \times 4 \text{ estrategias} \times 3 \text{ réplicas}$). Tras descartar réplicas con `parse_ok = 0`, cada pareja (tipo × N) se computa con una media de ≈ 85 réplicas válidas (rango 68–202; la pareja C_stratified · N=100 concentra más réplicas por su reutilización posterior en el banco principal). Este tamaño es el que sostiene todas las medias reportadas en el apartado 4.1.

4.1.2. Evaluación

Cada experimento se evalúa con los tres niveles del apartado 3.7: sintáctico (`parse_ok` tras post-procesado), estructural (OQuaRE Global y sus cinco subcaracterísticas + consistencia Hermit) y funcional (fidelidad léxica frente a las 88 URIs gold y cobertura de las 15 preguntas de competencia). El valor de cada celda es la media sobre sus réplicas válidas, con `n_runs OK`.

4.1.3. Selección de la pareja (tipo*, N*)

La pareja se elige con una regla lexicográfica fijada a priori: validez → OQuaRE Global → fidelidad léxica → cobertura CQ. La decisión se toma sobre gpt-4o, cuya validez es prácticamente total en toda la rejilla (Tabla 8): así la elección recae en la calidad estructural y funcional y no en la tasa de parseo, aislando el efecto del muestreo de la inestabilidad de los modelos compactos. El agregado de los tres modelos se reporta como comprobación de robustez.

Tabla 8 Sensibilidad conjunta al tipo de muestreo y al tamaño, sobre gpt-4o

(tipo, N)	validez	OQuaRE	canon	CQ
A_head · 25	1.00	3.48	0.34	0.25
A_head · 50 ‡	1.00	3.54	0.33	0.25
A_head · 100	0.98	3.63	0.57	0.33
A_head · 200	1.00	3.62	0.52	0.35
B_random · 25	1.00	3.37	0.33	0.23
B_random · 50	0.98	3.41	0.37	0.22
B_random · 100	1.00	3.65	0.63	0.36
B_random · 200	0.98	3.64	0.61	0.35
C_stratified · 25	0.95	3.38	0.35	0.23
C_stratified · 50	0.98	3.50	0.34	0.23
C_stratified · 100 ★	0.98	3.58	0.59	0.38
C_stratified · 200	1.00	3.65	0.58	0.36
D_diversity · 25	1.00	3.56	0.32	0.23
D_diversity · 50	0.98	3.51	0.41	0.26
D_diversity · 100	0.97	3.63	0.53	0.37
D_diversity · 200	0.95	3.66	0.58	0.35

Pareja seleccionada: C_stratified · N=100. En gpt-4o la validez es prácticamente total en toda la rejilla y el OQuaRE Global es casi plano (3,37–3,66), por lo que la decisión la marca el nivel funcional:

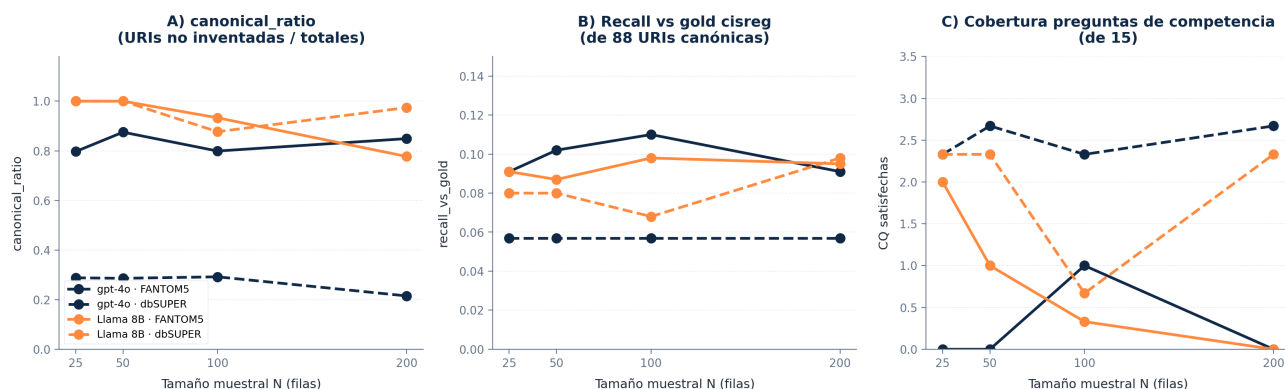
C_stratified a N=100 alcanza la mayor cobertura CQ (0,38) y una fidelidad léxica alta (canon 0,59), empatando en la práctica con las demás celdas de N≥100 y muestreo representativo. Se elige N=100 —y no N=200, indistinguible— por lograr el mismo perfil con la mitad de tokens. Esta pareja se congela para los apartados 4.2 y 4.3.

El agregado de los tres modelos selecciona A_head · N=50, distinto de la elección sobre gpt-4o, pero con diferencias igualmente pequeñas: el muestreo y el tamaño apenas mueven la calidad. Esta robustez es en sí misma un resultado —las conclusiones del banco no dependen críticamente de (tipo*, N*)— y justifica fijar estos parámetros para el resto de experimentos.

Las diferencias de OQuaRE Global entre C_stratified·100 y las demás cimas de gpt-4o son inferiores a 0,2 puntos, muy por debajo de la mínima diferencia detectable con n = 3 réplicas (≈1,7 puntos; Anexo C): son observacionales. Estadísticamente, las celdas de la parte alta de la Tabla 8 son indistinguibles, lo que confirma la robustez del banco frente a estos parámetros.

Figura 4 Sensibilidad al tamaño muestral N

Sensibilidad al tamaño muestral $N \in \{25, 50, 100, 200\}$
E3 RAG semántico · gpt-4o vs Llama 3.1 8B · FANTOM5 y dbSUPER



4.2. Calibración del sistema RAG (experimento E3)

E3 inyecta contexto recuperado por el RAG semántico (apartado 3.5.2). Su configuración por defecto —la empleada hasta aquí— es el baseline. Esta sección determina la mejor configuración del retrieval con el muestreo ya fijado en §4.1.

4.2.1. Diseño

Tres modelos (gpt-4o, Llama 3.1 8B, Qwen 2.5 Coder 7B) × cuatro configuraciones del recuperador × cuatro bases de datos, con (tipo*, N*) = (C_stratified, 100) fijado en §4.1 y tres réplicas por celda. La configuración baseline reproduce los tres parámetros del retrieval fijados en §3.5.2: top_k = 5 (fragmentos recuperados por consulta), score_thr = 0,4 (umbral mínimo de similitud coseno) y max_chars = 5000 (tope de caracteres del contexto inyectado). Las tres variantes son perturbaciones puntuales sobre ese baseline: C1 reduce top_k a 2 (menos fragmentos, contexto más selectivo, resto de parámetros igual que baseline); C2 sube score_thr a 0,6 (sólo fragmentos con similitud alta, resto igual); y C3 combina las dos anteriores (top_k = 2 y score_thr = 0,6). Calibrar sobre los tres modelos —y no sólo sobre gpt-4o— reduce el sesgo del referente único y permite documentar la dependencia del óptimo con el tamaño del modelo. Tamaño detrás de los valores: 3 modelos × 4 configuraciones × 4 BDs × 3 réplicas = 144 ejecuciones planificadas para la Tabla 9; cada celda (modelo × configuración)

agrega 12 ejecuciones planificadas (4 BDs × 3 réplicas), y tras descartar réplicas con parse_ok = 0 cada media reportada se calcula sobre ≈ 10 réplicas válidas por celda (rango 5–12; véase la columna «validez» de la Tabla 9).

4.2.2. Evaluación

Las mismas tres dimensiones del apartado 3.7, por celda (modelo, configuración, base de datos), con media sobre réplicas válidas y n_runs OK.

4.2.3. Resultados y selección de RAG*

En gpt-4o el baseline es la configuración óptima. Con el baseline, gpt-4o alcanza validez total (12/12) y el mejor OQuaRE Global entre las configuraciones plenamente válidas (4,19); C1 (top_k=2) eleva marginalmente el OQuaRE (4,24) pero pierde dos réplicas (10/12). Esto demuestra empíricamente, y ya no asume, que el baseline es la mejor elección para el modelo de referencia.

Tabla 9 Calibración del RAG en E3

modelo	config	validez	OQuaRE	canon	CQ
gpt-4o	baseline ★	12/12	4.19	0.95	0.28
gpt-4o	C1 (top_k=2)	10/12	4.24	0.95	0.30
gpt-4o	C2 (thr=0,6)	11/12	4.18	0.98	0.23
gpt-4o	C3	11/12	4.14	0.78	0.32
Llama 3.1 8B	baseline	11/12	1.00	1.00	0.04
Llama 3.1 8B	C1 (top_k=2) ▲	12/12	1.77	0.86	0.13
Llama 3.1 8B	C2 (thr=0,6)	12/12	1.67	0.86	0.07
Llama 3.1 8B	C3	6/12	1.00	0.90	0.07
Qwen 2.5 C.	baseline	5/12	3.27	1.00	0.30
Qwen 2.5 C.	C1 (top_k=2)	12/12	1.00	0.94	0.07
Qwen 2.5 C.	C2 (thr=0,6)	12/12	1.67	1.00	0.00
Qwen 2.5 C.	C3	9/12	1.00	1.00	0.00

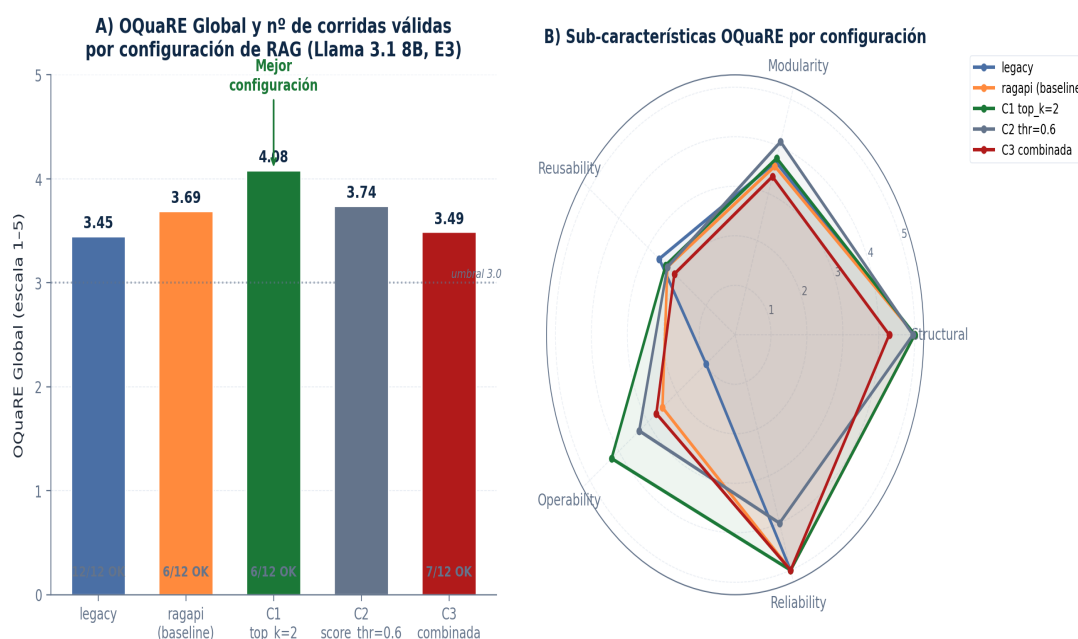
OQuaRE, canon y CQ son medias sobre las réplicas válidas de las cuatro bases. ★ configuración fijada para el banco (RAG*); ▲ óptimo del modelo compacto.

El óptimo depende del tamaño del modelo. En Llama 3.1 8B la relación se invierte: el baseline satura al modelo compacto —OQuaRE 1,00— y es C1 (top_k=2) la configuración que lo rescata (OQuaRE 1,77 con validez 12/12). Es decir, reducir el número de fragmentos recuperados beneficia a los modelos compactos y perjudica poco al cerrado. Este resultado, obtenido sobre los tres modelos y no solo sobre Llama, constituye la guía de calibración del RAG por tamaño de modelo que el trabajo aporta. Qwen 2.5 Coder se comporta de forma inestable entre configuraciones y no permite un óptimo claro.

Selección. Para el banco principal (§4.3) se fija una única configuración RAG* = baseline, la óptima sobre gpt-4o —coherente con el criterio de §4.1—, de modo que la comparativa cross-model de E3 se realiza con un retrieval idéntico para todos los modelos. La dependencia con el tamaño del modelo no se introduce como parámetro variable del banco, sino que se documenta como hallazgo y guía operativa. La significación de las diferencias entre configuraciones se detalla en el Anexo C. Nota: el OQuaRE de E3:gpt-4o-baseline es 4,19 en esta tabla (fase de calibración) y 3,91 en la Tabla 12 (banco

principal). Ambos resultados son consistentes con la clasificación de baseline como configuración óptima para gpt-4o (mayor OQuaRE en las configuraciones plenamente válidas).

Figura 5 Calibración del sistema RAG



4.3. Banco principal: triple validación con los parámetros fijados

Con (tipo*, N*, RAG*) congelados (§4.1–4.2), se ejecuta el banco completo: cuatro estrategias (E1–E4) × tres modelos × cuatro bases de datos × tres réplicas, reutilizando las celdas ya generadas con esos mismos parámetros. Todas las ontologías se evalúan con los **tres niveles** del apartado 3.6; los análisis posteriores —por estrategia, por modelo, por base de datos, cross-model y transversales— se construyen sobre este banco **único y homogéneo**. Es esta homogeneidad la que permite comparar entre sí estrategias, modelos y bases sin mezclar configuraciones.

4.3.1. Validación sintáctica y efecto del post-procesado

Se reporta parse_ok antes y después del post-procesado mecánico determinista (apartado 3.6). El post-procesado se aplica de forma idéntica a E1–E3 en los tres modelos; E4 se evalúa sobre la salida cruda de OntoGenix. Su efecto se contrasta con la prueba de McNemar sobre las discordancias por fichero.

Tabla 10 Parse_ok antes/después del post-procesado, por estrategia.

Estrategia	parse_ok antes	parse_ok después	McNemar
E1	15/36	15/36	— (sin discordancias)
E2	23/36	36/36	$\chi^2 = 11,1 \cdot p < 0,001$
E3	15/36	28/36	$\chi^2 = 11,1 \cdot p < 0,001$
E4	18/36 (crudo)	— (no aplica)	— (evaluado en crudo; gpt-4o 12, Qwen 6, Llama 0)

E1–E3: 3 modelos × 4 BD × 3 réplicas = 36 runs; los valores son tras post-procesado. E4: evaluado en crudo (gpt-4o 12/12, Qwen 6/12, Llama 0/12; no se aplica post-procesado).

El post-procesado rescata de forma significativa E2 (+13; $p < 0,001$) y E3 (+13; $p < 0,001$), pero no rescata ninguna réplica de E1: los fallos del zero-shot son mayoritariamente errores de fondo no reparables mecánicamente. La estadística de errores tipados por modelo (Tabla 11) revela la firma de error característica de cada uno.

Tabla 11 Errores sintácticos tipados por modelo (salida cruda, banco C_stratified-100)

Modelo	Ficheros	Prefijos no declarados	Prefijo desnudo
gpt-4o	48	3	0
Llama 3.1 8B	36	4	0
Qwen 2.5 Coder	48	57	6

Gpt-4o y Llama son sintácticamente limpios; Qwen concentra la mayoría de los errores de prefijo, lo que explica su fallo total en E1. En este banco no se observan serializaciones cruzadas ni alucinaciones tipadas.

NOTA: Un prefijo desnudo es un identificador que aparece en el Turtle sin la declaración @prefix por lo que el parser no puede resolverlo a una URI.

4.3.2. Validación estructural (OQuaRE + HerMiT)

La Tabla 12 da el OQuaRE Global medio y las réplicas válidas por (estrategia, modelo); todas las medias son sobre réplicas válidas.

Tabla 12 OQuaRE Global medio (válidas/total) por estrategia y modelo

	gpt-4o	Llama 3.1 8B	Qwen 2.5 Coder
E1	4,00 (12/12)	4,37 (3/12)	— (0/12)
E2	3,94 (12/12)	4,10 (12/12)	4,17 (12/12)
E3	3,91 (11/12)	1,00 (11/12)	3,27 (6/12)
E4	3,86 (12/12)	— (0/12) †	4,29 (6/12)

Banco C_stratified-100, RAG = baseline. † E4-Llama no produce ontologías válidas (§4.3.5).*

De los resultados mostrados en la Tabla 12 podemos hacer tres observaciones: E2 (vocabulario controlado) es fuerte y homogéneo entre modelos, los abiertos igualan o superan a gpt-4o (Llama 4,10 y Qwen 4,17 frente a 3,94, una inversión que se explica por las rúbricas OQuaRE (§5), además E3 con el baseline colapsa en el modelo compacto (Llama 1,00), coherente con la calibración de §4.2 (el baseline satura a los compactos) y por último, la consistencia lógica es total: el 100 % de las ontologías válidas pasan HerMiT (gpt-4o 47/47, Llama 26/26, Qwen 24/24). El desglose por subcaracterísticas OQuaRE (Structural, Modularity, Reusability, Operability, Reliability) por modelo se muestra en la Tabla 13.

Figura 6 OQuaRE Global por experimento y modelo

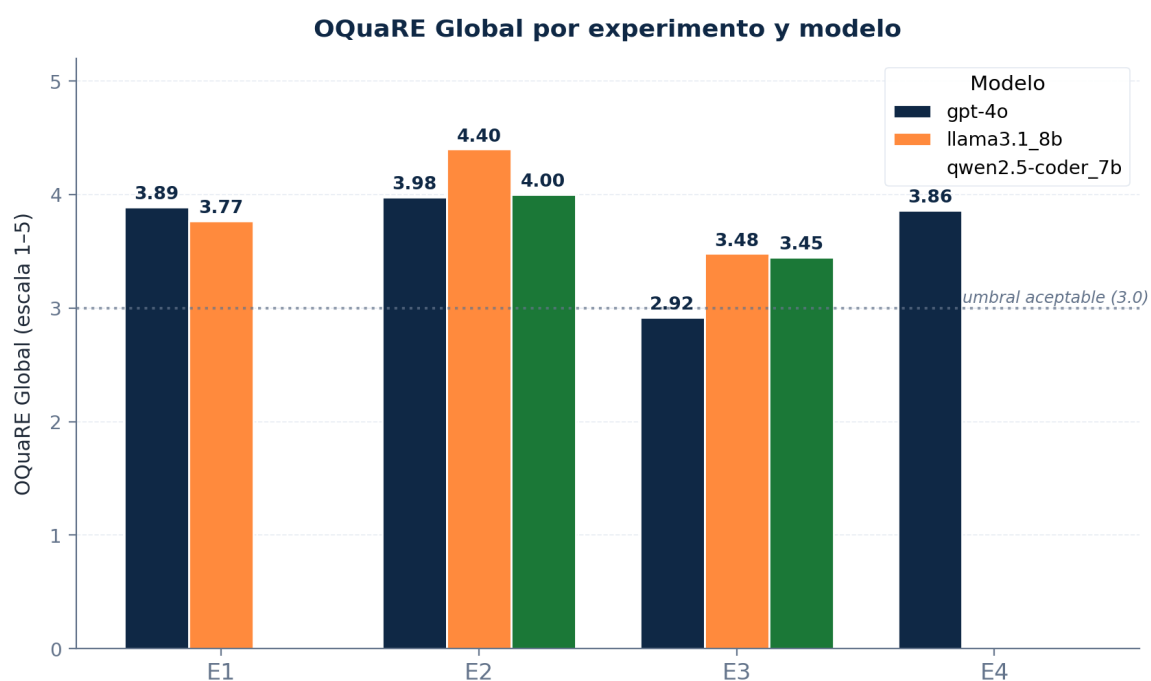
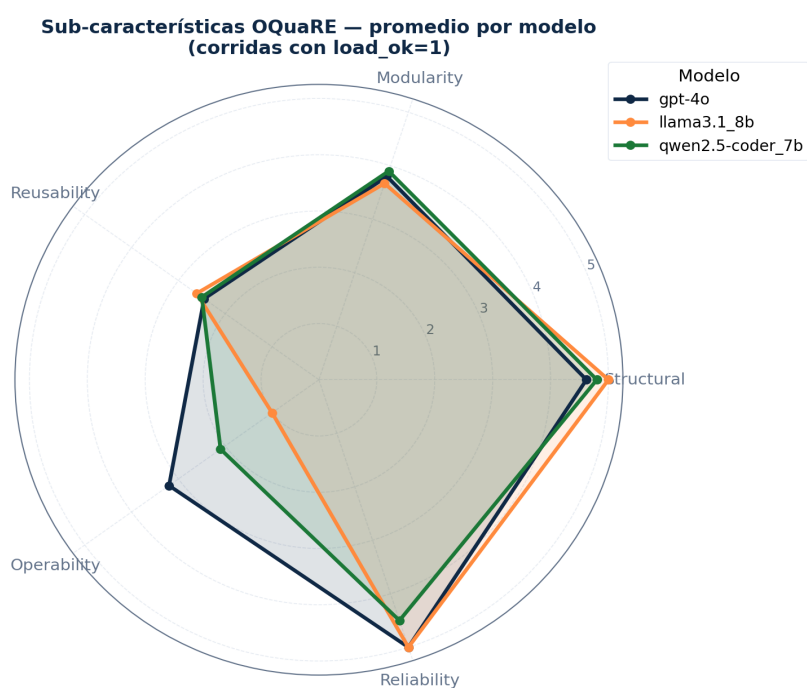


Tabla 13 Media de cada subcaracterística OQuaRE por modelo (réplicas válidas de E1–E4).

Modelo	Struct.	Modul.	Reusab.	Operab.	Reliab.
gpt-4o	4,89	3,63	2,38	3,72	5,00
Llama 3.1 8B	2,88	2,54	1,42	2,25	5,00
Qwen 2.5 Coder	5,00	4,00	2,41	3,44	5,00

Figura 7 Características OQuaRE por modelo



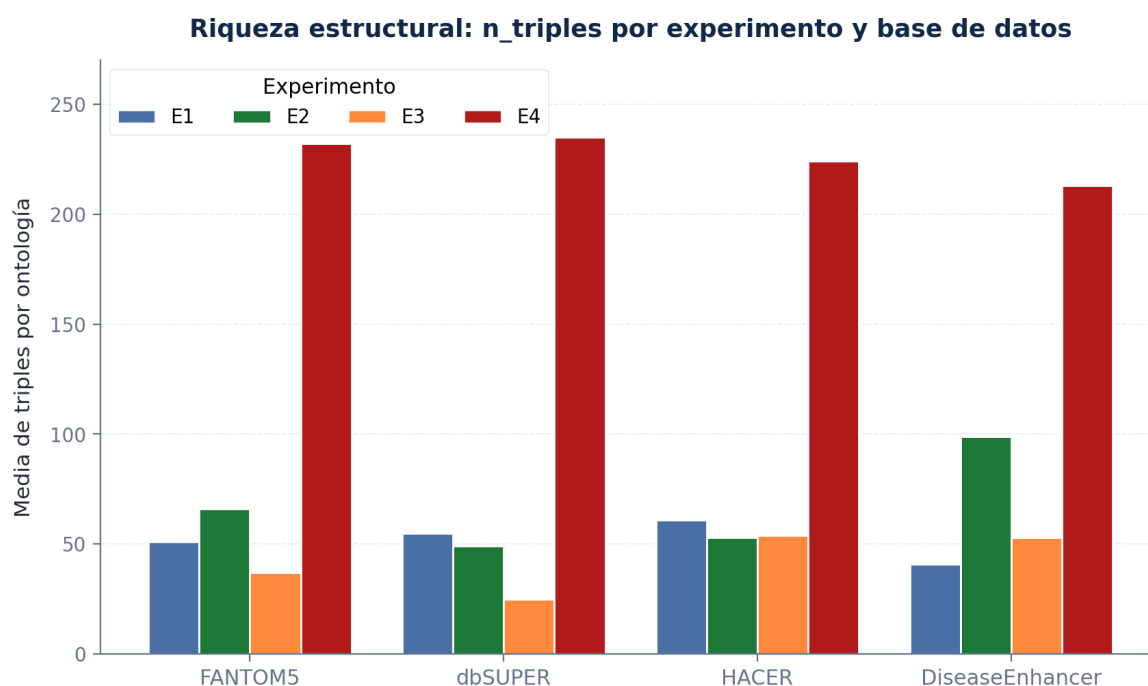
Densidad estructural (Tabla 14). Sobre gpt-4o, la media de tripletas por ontología es 49 (E1), 56 (E2), 53 (E3) y 226 (E4), con 10 clases y 28 propiedades de datos en E4 frente a 4–5 clases en el resto: E4 es $\approx 4\times$ más denso que las baselines de prompting, lo que sustenta la hipótesis H1.

Tabla 14 Densidad estructural por modelo y estrategia

Modelo · Estrategia	Tripletas/onto (media)	Clases (media)	Prop. objeto (media)	Prop. datos (media)	n_OK / total	Notas
gpt-4o · E1	55	4.2	2.5	5.2	12/12	—
gpt-4o · E2	63	4.9	3.2	5.7	12/12	vocab. controlado
gpt-4o · E3	38	2.3	0.0	0.3	12/12	RAG baseline
gpt-4o · E4 ★	225	9.6	6.2	27.0	12/12	★ máx densidad
Llama 3.1 8B · E1	72	2.0	10.0	0.0	3/12	baja validez
Llama 3.1 8B · E2	84	1.0	11.3	4.0	9/12	—
Llama 3.1 8B · E3	23	2.0	0.0	0.0	9/12	OQuaRE colapsa (1,00)
Llama 3.1 8B · E4	—	—	—	—	0/12	sin ontologías válidas
Qwen 2.5 Coder 7B · E1	—	—	—	—	0/12	sin ontologías válidas
Qwen 2.5 Coder 7B · E2	52	2.5	5.4	3.2	8/12	—
Qwen 2.5 Coder 7B · E3	27	3.3	2.3	0.0	9/12	—
Qwen 2.5 Coder 7B · E4	115	5.0	1.0	1.0	6/12	—

Tripletas, clases y propiedades (objeto y datos) por ontología, media sobre las réplicas válidas del banco *C_stratified*·100. — indica sin réplicas válidas para esa celda. ★ máxima densidad estructural.

Figura 8 Riqueza estructural n_triples medio



4.3.3. Validación funcional (fidelidad léxica + preguntas de competencia)

La fidelidad léxica reordena la lectura: **E2** fuerza el reuso de vocabulario controlado en Qwen (canonical_ratio 1,00) pero no en gpt-4o (canon 0,48, inferior al 0,94 de E1); con recall bajo frente al gold en todos los modelos.

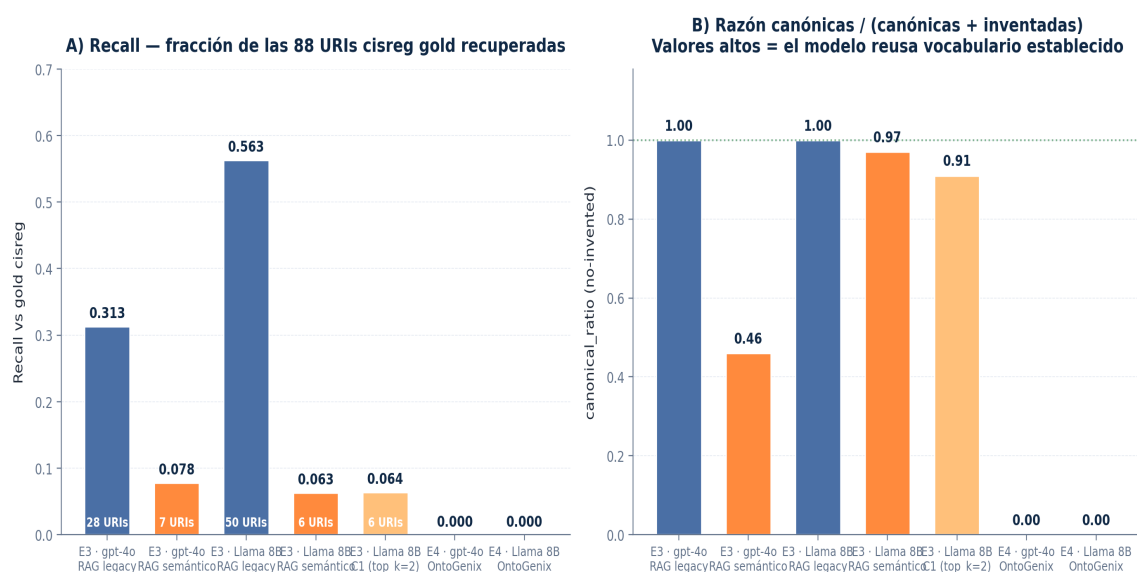
Tabla 15 Fidelidad léxica y cobertura CQ, por estrategia y modelo.

Estrategia · modelo	canon	recall	CQ	n_OK
E1 · gpt-4o	0,94	0,016	0,32	12/12
E1 · Llama	0,44	0,011	0,07	3/12
E2 · gpt-4o	0,48	0,012	0,34	12/12
E2 · Llama	0,08	0,011	0,15	12/12
E2 · Qwen	1,00	0,011	0,13	12/12
E3 · gpt-4o	0,94	0,104	0,31	11/12
E3 · Llama	1,00	0,072	0,03	11/12
E3 · Qwen	1,00	0,068	0,30	6/12
E4 · gpt-4o	0,00	0,00	0,66	12/12
E4 · Qwen	0,00	0,00	0,23	6/12

E3 semántico logra el mejor recall en gpt-4o (0,104) manteniendo canon alto; y **E4**, pese a la mayor cobertura funcional (CQ 0,66 en gpt-4o), declara todas sus URIs en namespaces ad-hoc (canon 0,00, recall 0,00). Ninguna configuración supera ≈0,66 de cobertura CQ: el LLM produce un primer borrador útil, no una ontología final.

Figura 9 Fidelidad al vocabulario cis regulatorio

Fidelidad léxico-semántica al dominio cisreg
(reuso de URIs canónicas vs URIs inventadas)



E3 logra el mejor recall en gpt-4o; E2 fuerza canon alto en Qwen.

4.3.4. Comparativa cross-model

La Tabla 16 sintetiza el perfil cross-model; los datos detallados por estrategia están en la Tabla 12 (OQuaRE) y la Tabla 15 (funcional). Esta comparativa responde a PI2: no existe una estrategia con ranking estable entre modelos. E2 es la más homogénea y la única en que los tres modelos superan el umbral aceptable; E3 con baseline discrimina fuertemente por capacidad (gpt-4o 3,91 frente a Llama 1,00); y en validez sintáctica gpt-4o es el único modelo válido en las cuatro estrategias (Qwen falla por completo en E1 —0/12—, Llama no produce E4). La aproximación de un modelo abierto al cerrado depende de la estrategia (E2) y, en E3, de la calibración del retrieval (§4.2). Las diferencias inferiores a ≈0,5 puntos (p. ej. la inversión de E2) se leen como observacionales (Anexo C).

Tabla 16 Síntesis cross-model: validez, OQuaRE, canon y cobertura CQ medias por modelo

Modelo	Validez agregada	OQuaRE (media)	Canon (media)	CQ (media)	Estrategias válidas	Observación clave
gpt-4o	47/48	3,93	0,59	0,41	E1, E2, E3, E4	Único modelo válido en las 4 estrategias; máx. CQ con E4 (0,66)
Qwen 2.5 Coder	24/48	3,91	0,67	0,24	E2, E3, E4	Falla E1 (0/12); en E2 iguala a gpt-4o en OQuaRE; inestable en E3
Llama 3.1 8B	26/48	3,16 *	0,51	0,08	E1 (3/12), E2, E3	No produce E4; E3 colapsa OQuaRE=1,00 con RAG*=baseline

Medias sobre réplicas válidas y estrategias aplicables. * Media de E1(3v), E2(12v), E3(11v); E4 excluido (0 válidas en Llama).

4.3.5. E4 (OntoGenix) con modelos open-source

Replicación de la pipeline multi-agente con los modelos abiertos frente a gpt-4o, para separar el efecto del método del efecto del modelo. Los resultados por modelo se muestran en la Tabla 17.

Tabla 17. E4 (OntoGenix) por modelo, banco C_stratified-100.

Métrica	gpt-4o	Qwen 2.5 C.	Llama 3.1 8B
Validez (load_ok)	12/12	6/12	0/12
Tripletas / ontología	≈ 226	parcial	n.d.
Clases / prop. datos	10 / 28	—	n.d.
Consistencia Hermit	100 %	100 %	n.d.
OQuaRE Global	3,86	4,29	n.d.
canon / recall	0,00 / 0,00	0,00 / 0,00	n.d.

E4 con gpt-4o produce las ontologías más densas (≈226 tripletas) y consistentes, pero con vocabulario ad-hoc; Qwen rescata 6/12; y **Llama 3.1 8B no genera ninguna ontología válida (0/12)**. El método multi-agente, por tanto, **no garantiza por sí solo** la validez de un modelo compacto: se necesitan un buen pipeline y un modelo capaz de respetar el formato.

4.3.6. Análisis transversales

(a) **Consistencia inter-BD.** Coeficiente de Jaccard medio entre los conjuntos de URIs de las cuatro bases dentro de cada (estrategia, modelo): valores altos = «copiador» (mismo núcleo reutilizado), bajos = «generador adaptativo» (Tabla 18).

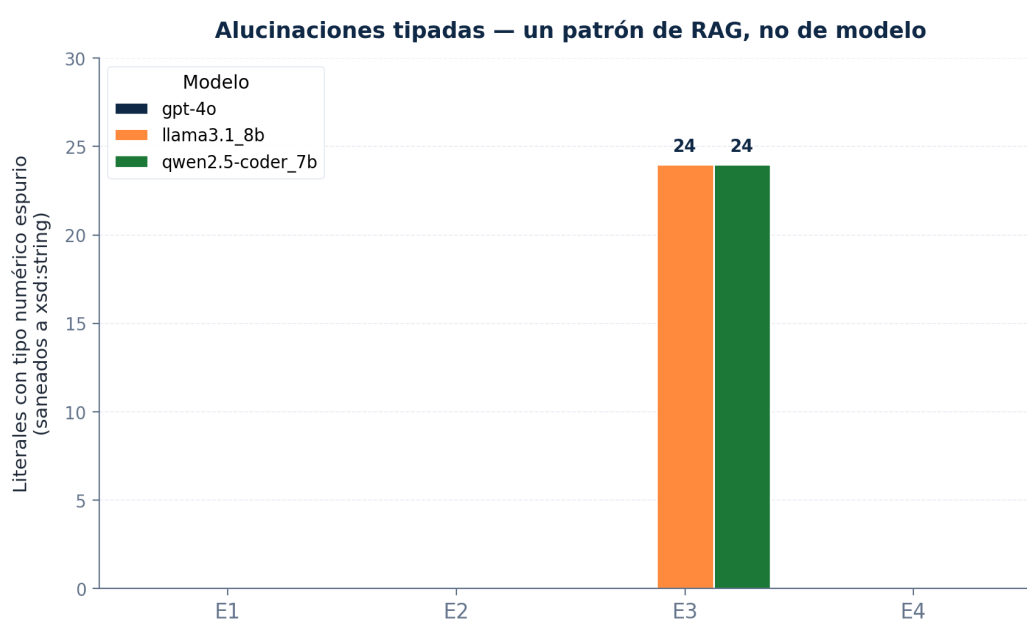
Tabla 18 Jaccard medio inter-BD por estrategia y modelo.

	gpt-4o	Llama 3.1 8B	Qwen 2.5 Coder
E1	0,41	0,22	0,61
E2	0,39	0,77	0,56
E3	0,27	0,30	0,14
E4	0,78	n.d.	0,46

E3 (RAG semántico) genera el núcleo más **adaptativo** (Jaccard 0,14–0,30, distinto por base), mientras que E2 en Llama (0,77) y E4 en gpt-4o (0,78) reutilizan un esqueleto casi idéntico entre bases («copiadores»). La elección de estrategia depende, pues, del objetivo: reutilización de vocabulario establecido frente a novedad estructural.

(b) **Patrones de error por modelo.** La firma de error (Tabla 11) es clara: Qwen 2.5 Coder concentra los errores de prefijo —prefijos usados sin declarar y sujetos con prefijo desnudo—, lo que explica su fallo completo en E1; gpt-4o y Llama son sintácticamente limpios. En este banco, con muestras representativas de 100 filas, no se detectan serializaciones cruzadas ni alucinaciones tipadas, a diferencia de lo observado en configuraciones previas con muestras más pequeñas. La significación de las afirmaciones principales se desarrolla en el Anexo C.

Figura 10 Alucinaciones terminológicas por estrategia y modelo



E2 minimiza alucinaciones gracias al vocabulario controlado.

5. Discusión

5.1. Reproducibilidad implementada

El banco se acompaña de un marco de reproducibilidad en tres niveles: manifiestos SHA-256 de las entradas deterministas, versiones fijadas de las dependencias críticas y snapshots de modelo (gpt-4o-2024-05-13 y digests de los modelos abiertos). Cada afirmación es reproducible al nivel que su naturaleza permite (estricto, estadístico, metodológico).

5.2. Discusión por pregunta de investigación

5.2.1. PI1 — ¿de qué depende el resultado: modelo, estrategia o estructura de los datos?

El resultado depende de la interacción de los tres factores. La **estrategia** explica la varianza en las métricas absolutas (densidad y riqueza: E4 $\approx$ 226 tripletas frente a 49–56 de las baselines); el **modelo**, la validez sintáctica (gpt-4o válido en las cuatro estrategias, Qwen 0/12 en E1); y la **estructura de los datos**, sobre todo en los modelos compactos. El análisis de sensibilidad conjunta (§4.1) añade un matiz importante: dentro del rango estudiado, el tipo de muestreo y el tamaño tienen un efecto pequeño y el banco es robusto frente a ellos.

5.2.2. PI2 — ¿existe una estrategia estable entre modelos?

No existe un ranking estable. **E2** (vocabulario controlado) es la estrategia más homogénea —la única en que los tres modelos superan el umbral aceptable—, con una inversión aparente en que los abiertos igualan a gpt-4o por efecto de las rúbricas OQuaRE. **E3** depende críticamente de la capacidad del modelo y de la calibración del retrieval: con el baseline el modelo compacto colapsa (Llama 1,00) y reducir top_k (C1) lo rescata. Solo gpt-4o es válido en las cuatro estrategias.

5.2.3. PI3 — dimensiones de calidad y métricas complementarias

Las cuatro dimensiones no son redundantes. OQuaRE premia la estructura pero penaliza la sobre-estructuración: E4, la más densa, queda intermedia (3,86). La fidelidad léxica reordena la lectura —E2 fuerza el reuso, E3 mantiene canon alto con el mejor recall, y E4 inventa todas sus URIs (canon 0)—, y la cobertura de preguntas de competencia fija un **techo empírico $\approx$ 0,66**. Las conclusiones se sostienen sobre la combinación de las cuatro dimensiones y no sobre OQuaRE Global aislado; cuestionar las rúbricas, y no solo los resultados, es una lección transferible.

5.2.4. PI4 — recuperabilidad mecánica y patrones de error

El post-procesado determinista separa el problema sintáctico del semántico y rescata E2 y E3 ($p < 0,001$), pero **no E1**: los fallos del zero-shot son errores de fondo. Cada modelo presenta una firma de error reproducible (Qwen mezcla serializaciones, Llama omite el «:» del prefijo y produce sujetos desnudos), y la interacción entre el RAG y los modelos compactos genera alucinaciones tipadas.

5.3. Lecciones metodológicas transferibles

Del trabajo se desprenden cinco aprendizajes: reportar simultáneamente la media y el número de réplicas válidas; diseñar el post-procesado antes del experimento; aislar los entornos de los servicios externos; cuestionar las rúbricas normativas (comprender qué propiedad optimizan).

5.4. Limitaciones

Con $n = 3$ réplicas por celda, la mínima diferencia OQuaRE detectable es $\approx 1,7$ puntos, por lo que muchas diferencias se reportan como observacionales (Anexo C). No se dispone de **ground truth** biomédico: las ontologías se comparan entre sí, no contra una de referencia, de modo que una puntuación alta no garantiza corrección del modelado. La pipeline E4 está diseñada para el modelo cerrado y no produce ontologías válidas con Llama 3.1 8B. Y los hallazgos cuantitativos son específicos del dominio cis-regulatorio.

5.5. Líneas de trabajo futuro

Cuatro líneas: validación experta sobre un subconjunto estratificado (comparar el ranking experto con el de OQuaRE); replicar E4 con un modelo abierto de mayor tamaño (Llama 3.1 70B) para aislar el efecto «método» del «modelo»; generalizar el banco fuera del dominio cis-regulatorio; y un sistema RAG híbrido léxico-semántico que combine la riqueza estructural del RAG semántico con la fidelidad léxica del vocabulario controlado.

6. Conclusiones

Este Trabajo de Fin de Máster ha desarrollado un banco experimental reproducible para evaluar cuatro estrategias de generación automática de ontologías biomédicas mediante modelos de lenguaje grandes, aplicado al dominio cis-regulatorio humano y organizado en una secuencia lógica que fija primero los parámetros de muestreo y de retrieval y después ejecuta el banco completo con una evaluación uniforme de tres niveles. La conclusión central es que la calidad de la ontología generada no depende de un único factor, sino de la **interacción entre el modelo, la estrategia de inyección de contexto y la estructura de los datos**, por lo que ninguna estrategia resulta universalmente óptima.

Respecto a las preguntas de investigación: el tipo de muestreo y el tamaño de muestra tienen un efecto pequeño y el banco es robusto frente a ellos. No existe una estrategia con ranking estable entre modelos, si bien la **calibración del RAG** (reducir top_k en los modelos compactos) acerca a Llama al modelo cerrado, y se demuestra empíricamente que para gpt-4o el baseline es la configuración óptima (PI2); las cuatro dimensiones de calidad no son redundantes y fijan un techo funcional $\approx 0,66$ de cobertura de preguntas de competencia (PI3); y el post-procesado mecánico rescata los fallos recuperables (E2/E3, $p < 0,001$) pero no los de fondo (E1), con firmas de error propias de cada modelo (PI4).

El trabajo deja tres aportaciones replicables: una **guía de calibración del RAG por tamaño de modelo** (baseline para el modelo cerrado, top_k reducido para los compactos, demostrada sobre los tres modelos); un **módulo de post-procesado mecánico** que separa el problema sintáctico del semántico sin nuevas llamadas al modelo; y una **métrica complementaria de fidelidad léxica** acompañada de un corpus de 15 preguntas de competencia, que discrimina entre reuso de vocabulario y generación libre. Todo ello se acompaña de un marco de reproducibilidad con garantías explícitas.

Referencias bibliográficas

- Andersson, R., Gebhard, C., Miguel-Escalada, I., Hoof, I., Bornholdt, J., Boyd, M., Chen, Y., et al. (2014). An atlas of active enhancers across human cell types and tissues. *Nature*, 507(7493), 455–461.
- Babaei Giglou, H., D’Souza, J., & Auer, S. (2023). LLMs4OL: Large Language Models for Ontology Learning. In *The Semantic Web – ISWC 2023* (LNCS 14265, pp. 408–427). Springer.
- Bandeira, J., Bittencourt, I. I., Espinheira, P., & Isotani, S. (2016). FOCA: A methodology for ontology evaluation. *arXiv:1612.03353*.
- Begley, C. G., & Ellis, L. M. (2012). Drug development: Raise standards for preclinical cancer research. *Nature*, 483(7391), 531–533.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., et al. (2020). Language models are few-shot learners. *NeurIPS*, 33, 1877–1901.
- Buehler, M. J. (2024). Generative retrieval-augmented ontologic graph and multiagent strategies for interpretive large language model-based materials design. *ACS Engineering Au*, 4(2).
- Caufield, J. H., Hegde, H., Emonet, V., Harris, N. L., Joachimiak, M. P., Matentzoglou, N., et al. (2024). Structured prompt interrogation and recursive extraction of semantics (SPIRES): a method for populating knowledge bases using zero-shot learning. *Bioinformatics*, 40(3), btac104.
- Cuenca-Grau, B., Horrocks, I., Motik, B., Parsia, B., Patel-Schneider, P., & Sattler, U. (2008). OWL 2: The next step for OWL. *Journal of Web Semantics*, 6(4), 309–322.
- Duque-Ramos, A., Fernández-Breis, J. T., Stevens, R., & Aussenac-Gilles, N. (2011). OQuaRE: A SQuaRE-based approach for evaluating ontology quality. *JRPIT*, 43(2), 159–176.
- Duque-Ramos, A., Boeker, M., Jansen, L., Schulz, S., Iniesta, M., & Fernández-Breis, J. T. (2014). Evaluating the GoodOD guideline with OQuaRE. *PLoS ONE*, 9(8), e104463.
- Fathallah, N., Das, A., De Giorgis, S., Poltronieri, A., Haase, P., & Kovriguina, L. (2024). NeOn-GPT: A large language model-powered pipeline for ontology learning. In *The Semantic Web: ESWC 2024 Satellite Events*. Springer.
- Glimm, B., Horrocks, I., Motik, B., Stoilos, G., & Wang, Z. (2014). Hermit: An OWL 2 reasoner. *Journal of Automated Reasoning*, 53(3), 245–269.
- Grüninger, M., & Fox, M. S. (1995). Methodology for the design and evaluation of ontologies. *IJCAI-95 Workshop on Basic Ontological Issues*.
- Gu, Y., Tinn, R., Cheng, H., Lucas, M., Usuyama, N., Liu, X., et al. (2021). Domain-specific language model pretraining for biomedical NLP. *ACM TOCH*, 3(1), 1–23.
- Gundersen, O. E., & Kjensmo, S. (2018). State of the art: Reproducibility in artificial intelligence. *AAAI*, 32(1).
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. *AAAI*, 32(1).
- Hui, B., Yang, J., Cui, Z., et al. (2024). Qwen2.5-Coder Technical Report. *arXiv:2409.12186*.
- Hutson, M. (2018). Artificial intelligence faces reproducibility crisis. *Science*, 359(6377), 725–726.
- Khan, A., & Zhang, X. (2016). dbSUPER: A database of super-enhancers in mouse and human genome. *Nucleic Acids Research*, 44(D1), D164–D171.
- Kommineni, V. K., König-Ries, B., & Samuel, S. (2024). From human experts to machines: An LLM-supported approach to ontology and knowledge graph construction. *arXiv:2403.08345*.
- Lamy, J. B. (2017). Owlready: Ontology-oriented programming in Python with automatic classification. *Artificial Intelligence in Medicine*, 80, 11–28.

- Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., & Kang, J. (2020). BioBERT: A pre-trained biomedical language representation model. *Bioinformatics*, 36(4), 1234–1240.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 33, 9459–9474.
- Liu, F., Shareghi, E., Meng, Z., Basaldella, M., & Collier, N. (2021). Self-alignment pretraining for biomedical entity representations. *NAACL-HLT*, 4228–4238.
- Martínez-Romero, M., Jonquet, C., O'Connor, M. J., Graybeal, J., Pazos, A., & Musen, M. A. (2017). NCBO Ontology Recommender 2.0. *Journal of Biomedical Semantics*, 8(1), 21.
- Mateiu, P. M., & Groza, A. (2023). Ontology engineering with large language models. *Procedia Computer Science*, 225, 1095–1104.
- Meta AI. (2024). The Llama 3 herd of models. <https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>
- Meyer, L. P., Stadler, C., Frey, J., Radtke, N., Junghanns, K., Meissner, R., et al. (2023). LLM-assisted knowledge graph engineering: Experiments with ChatGPT. *arXiv:2307.06917*.
- Mihindukulasooriya, N., Tiwari, S., Enguix, C. F., & Lata, K. (2023). Text2KGBench: A benchmark for ontology-driven knowledge graph generation from text. In *The Semantic Web – ISWC 2023*. Springer.
- Motik, B., Patel-Schneider, P. F., & Parsia, B. (Eds.). (2012). *OWL 2 Web Ontology Language*. W3C Recommendation.
- Nayeri, M., Yogi, A. A., Fathallah, N., Thapa, R. B., Tautenhahn, H.-M., Schnurpel, A., & Staab, S. (2025). Retrieval-augmented generation of ontologies from relational databases. *arXiv:2506.01232*.
- Neuhaus, F. (2023). Ontologies in the era of large language models — A perspective. *Applied Ontology*, 18(4), 399–407.
- Norouzi, S. S., Mahdavinejad, M. S., & Hitzler, P. (2024). Conversational ontology alignment with ChatGPT. *arXiv:2308.09217*.
- Ochs, C., Geller, J., Perl, Y., Chen, Y., Xu, J., Min, H., & Case, J. T. (2017). Scalable quality assurance for large SNOMED CT hierarchies. *JAMIA*, 24(3), 524–533.
- Ollama. (2024). Ollama: Get up and running with large language models locally. <https://ollama.com>
- Open Science Collaboration. (2015). Estimating the reproducibility of psychological science. *Science*, 349(6251), aac4716.
- OpenAI. (2024). GPT-4o system card. <https://openai.com/index/gpt-4o-system-card/>
- Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., & Larochelle, H. (2021). Improving reproducibility in machine learning research. *JMLR*, 22(164), 1–20.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *EMNLP 2019*, 3982–3992.
- Reynolds, L., & McDonell, K. (2021). Prompt programming for large language models. *CHI Extended Abstracts*.
- Riquelme-García, A., Mulero-Hernández, J., & Fernández-Breis, J. T. (2025). annotationRAG: RAG-based multi-ontology annotation framework [software]. Tecnomod, Universidad de Murcia. <https://github.com/tecnomod/annotationRAG>
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Studer, R., Benjamins, V. R., & Fensel, D. (1998). Knowledge engineering: Principles and methods. *Data & Knowledge Engineering*, 25(1–2), 161–197.

- Tartir, S., Arpinar, I. B., Moore, M., Sheth, A. P., & Aleman-Meza, B. (2005). OntoQA: Metric-based ontology quality analysis. *IEEE ICDM Workshop*.
- Val-Calvo, M., Egaña Aranguren, M., Mulero-Hernández, J., Almagro-Hernández, G., Deshmukh, P., Bernabé-Díaz, J. A., Espinoza-Arias, P., Sánchez-Fernández, J. L., Mueller, J., & Fernández-Breis, J. T. (2025). OntoGenix: Leveraging large language models for enhanced ontology engineering from datasets. *Information Processing & Management*, 62(3), 104042.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 5998–6008.
- Vrandečić, D. (2010). Ontology evaluation. In S. Staab & R. Studer (Eds.), *Handbook on ontologies* (pp. 293–313). Springer.
- W3C. (2014). RDF 1.1 Turtle: Terse RDF Triple Language. W3C Recommendation.
- Wang, J., Dai, X., Shen, X., Ge, X., Wei, B., Tian, R., et al. (2019). HACER: An atlas of human active enhancers to interpret regulatory variants. *Nucleic Acids Research*, 47(D1), D106–D112.
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained Transformers. *NeurIPS*, 33, 5776–5788.
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Zhang, G., Shi, J., Zhu, S., Lan, Y., Xu, L., Li, H., et al. (2018). DiseaseEnhancer: A resource of human disease-associated enhancer catalog. *Nucleic Acids Research*, 46(D1), D78–D84.

Anexo A. Descripción de columnas de las bases de datos

Tras la homogeneización descrita en el apartado 3.1, las cuatro bases de datos del corpus (FANTOM5, dbSUPER, HACER, DiseaseEnhancer) comparten un esquema de treinta columnas canónicas con idéntico formato y orden. La tabla siguiente reproduce el significado de cada columna. Las columnas marcadas con (opt.) pueden contener valores vacíos en alguna base de datos, codificados como «-».

Columna	Tipo	Significado
CRM_ID	string	Identificador único del módulo cis-regulador.
chromosome	string	Cromosoma humano (chr1–chr22, chrX, chrY).
start_hg19 / end_hg19	int	Coordenadas genómicas en el ensamblaje hg19.
start_hg38 / end_hg38	int	Coordenadas genómicas en el ensamblaje hg38.
strand	char	Sentido de la hebra (+ / - / .).
score	float	Score de confianza del CRM (escala dependiente de la fuente).
pubmed_id (opt.)	int	Referencia PubMed asociada.
biosample / biosample_name	string	Tejido o línea celular en la que se detectó.
evidence_method	string	Método experimental (CAGE, ChIP-seq, eQTL, etc.).
regulation_type	string	Tipo de regulación (enhancer, super-enhancer, etc.).
source_db	string	Base de datos de origen (constante en cada fichero).
target_genes (opt.)	list	Lista de genes diana, separados por «;».
transcription_factors (opt.)	list	Lista de TF que se unen al CRM.
disease (opt.)	string	Enfermedad asociada (OMIM, ICD).
mutation (opt.)	list	Mutaciones registradas en el CRM.
cell_type (opt.)	string	Tipo celular específico cuando difiere de biosample.
tissue (opt.)	string	Tejido cuando difiere de biosample.
activity_score (opt.)	float	Score de actividad transcripcional (HACER, FANTOM5).
target_distance (opt.)	int	Distancia del CRM a su gen diana (bp).
evidence_level (opt.)	string	Nivel de evidencia (high/medium/low).
assembly_version	string	Versión del ensamblaje (hg19/hg38) usada como primario.
last_updated	date	Fecha de la última actualización del registro.
external_xref (opt.)	list	Referencias cruzadas a otras BD (UniProt, Ensembl, etc.).
pathway (opt.)	list	Rutas biológicas asociadas (KEGG, Reactome).
sex (opt.)	char	Sexo de la muestra (M/F/U).
age (opt.)	int	Edad de la muestra (años).
disease_stage (opt.)	string	Estadio de la enfermedad (si aplica).
snp_id (opt.)	string	Identificador SNP (rsID).

go_terms (opt.)	list	Términos GO asociados.
-----------------	------	------------------------

Anexo B. Abreviaturas y siglas

Sigla	Significado
ABox	Assertion Box · capa de instancias en OWL
API	Application Programming Interface
BD	Bases de datos
CQ	Competency Questions · preguntas de competencia
CRM	Cis-Regulatory Module
CSV/TSV	Comma/Tab Separated Values
FANTOM5	Functional Annotation of the Mammalian Genome 5
GUI	Graphical User Interface
HACER	Human ACTIVE Enhancer to gene Regulation
HermiT	Razonador OWL 2 DL
IRI	Internationalized Resource Identifier
ISO/IEC 25000	Estándar de calidad de software (SQuaRE)
LLM	Large Language Model
OQuaRE	Ontology Quality Requirements and Evaluation
OWL 2	Web Ontology Language 2
RAG	Retrieval-Augmented Generation
RDF	Resource Description Framework
R1 / R2 / R3	Reproducibilidad estricta / estadística / metodológica
SPARQL	SPARQL Protocol and RDF Query Language
TBox	Terminology Box · capa de clases y propiedades
TFM	Trabajo de Fin de Máster
TTL	Turtle, serialización RDF
URI	Uniform Resource Identifier

Las once métricas OQuaRE empleadas (WMConTo, NOMOnTo, DITM, NACOnTo, CBOnTo, TMOnto, LCOMOnTo, ANOnTo, INROnTo, RROnto y AROnTo) se documentan con su rúbrica completa en la Tabla 6 del cuerpo de la memoria.

Anexo C. Tests estadísticos: descripción, justificación y limitaciones

Las afirmaciones cuantitativas del Capítulo 4 se acompañan de una capa de inferencia cuyo fin es distinguir las diferencias reales de las atribuibles al azar. El diseño impone dos restricciones que condicionan toda esta capa y conviene declarar de entrada: (i) el número de réplicas por celda es pequeño ($n = 3$ semillas), lo que limita la potencia; y (ii) las métricas (OQuaRE, fidelidad, cobertura) no siguen necesariamente una distribución normal. Por ello se emplean métodos **no paramétricos y de remuestreo**, y la significación se interpreta con cautela: muchas diferencias se reportan como *observacionales* y no como estadísticamente significativas. Las comparaciones se hacen **pareadas** siempre que las observaciones comparten semilla y base de datos, para controlar la variabilidad entre celdas.

C.1. Bootstrap pareado de la diferencia de medias

Qué es. Remuestreo con reemplazo (10 000 réplicas) que estima la distribución de la diferencia de medias entre dos configuraciones y construye un intervalo de confianza al 95 % por percentiles, sin asumir normalidad.

Por qué se aplica. Para cuantificar diferencias de OQuaRE Global y de otras métricas continuas entre estrategias o configuraciones, con un IC interpretable cuando el tamaño muestral es pequeño y la distribución es desconocida.

Limitaciones con estos datos. Con $n = 3$ observaciones pareadas el remuestreo dispone de muy poca información: los IC son anchos y su cobertura nominal es solo aproximada. Se usa como estimación de la magnitud y la dirección del efecto, no como prueba de significación estricta.

C.2. Wilcoxon de rangos con signo (pareado)

Qué es. Prueba no paramétrica para dos muestras pareadas que contrasta si la mediana de las diferencias es cero, basada en rangos.

Por qué se aplica. Alternativa robusta a la t pareada cuando no se asume normalidad, para comparaciones pareadas con suficientes pares ($n \geq 6$), p. ej. la densidad estructural de E4 frente a las baselines agregando celdas.

Limitaciones con estos datos. Requiere un mínimo de pares para tener potencia y un p-valor fiable; por debajo de ese umbral (comparaciones por celda con $n = 3$) no se aplica o solo se usa su aproximación normal con cautela. Es sensible a los empates.

C.3. McNemar (proporciones pareadas)

Qué es. Prueba para datos binarios pareados que contrasta si cambia la proporción de éxitos entre dos condiciones sobre los mismos sujetos (tabla 2×2 de discordancias).

Por qué se aplica. Es el test adecuado para el efecto del post-procesado sobre parse_ok: cada ontología es éxito/fallo antes y después sobre el mismo fichero, de modo que las observaciones están pareadas por construcción.

Limitaciones con estos datos. Informa de la dirección y significación del cambio en la tasa de validez, no de su magnitud estructural; con pocas discordancias su versión asintótica es imprecisa y conviene la versión exacta.

C.4. Tamaño de efecto (Cohen's d)

Qué es. Medida estandarizada de la magnitud de una diferencia (diferencia de medias dividida por la desviación típica), independiente del tamaño muestral.

Por qué se aplica. Para complementar los p-valores con la relevancia práctica de las diferencias, ya que con n pequeño un efecto grande puede no alcanzar significación; conviene reportar magnitud y significación juntas.

Limitaciones con estos datos. Con $n = 3$ la estimación de la desviación típica es inestable, por lo que d debe leerse como orden de magnitud; los valores muy altos reflejan diferencias grandes pero con incertidumbre amplia.

C.5. Análisis de potencia y mínima diferencia detectable

Qué es. Cálculo de la diferencia mínima que el diseño puede detectar con una potencia dada (80 %) y $\alpha = 0,05$ bilateral, a partir de la desviación típica muestral.

Por qué se aplica. Para explicitar el techo inferencial del banco y justificar por qué muchas diferencias se reportan como observacionales.

Limitaciones / resultado. La desviación típica muestral de OQuaRE Global en el banco es 1,03 ($n = 97$ réplicas válidas). Con $n = 3$ réplicas por celda, solo son detectables diferencias superiores a $\approx 1,7$ puntos (Tabla 19); por debajo se declaran observacionales.

Tabla 19 Mínima diferencia detectable

n por celda	Δ mínima detectable (OQuaRE)
3	1,67
6	1,18
12	0,83
24	0,59

Mínima diferencia de OQuaRE Global detectable al 80 % de potencia ($\alpha = 0,05$ bilateral), con SD muestral = 1,03, por tamaño de celda.

En consecuencia, superan el umbral y son significativas el efecto del post-procesado (McNemar E2/E3, $p < 0,001$; Tabla 10) y la densidad de E4 ($\approx 4\times$ las baselines); mientras que las inversiones de E2 entre modelos (diferencias $< 0,5$) quedan como observacionales.

C.6. Corrección por comparaciones múltiples (Bonferroni-Holm)

Qué es. Procedimiento secuencial que controla la tasa de error de tipo I cuando se realizan varias pruebas, menos conservador que Bonferroni simple.

Por qué se aplica. El Capítulo 4 contrasta múltiples afirmaciones; sin corrección, la probabilidad de al menos un falso positivo entre todas crece. Holm-Bonferroni controla esa probabilidad familiar conservando más potencia que Bonferroni.

Limitaciones con estos datos. Sigue siendo conservador: afirmaciones de efecto moderado pueden perder significación tras corregir. Se reporta el p-valor antes y después de la corrección para transparencia.

C.7. Síntesis de limitaciones

En conjunto: $n = 3$ réplicas por celda implica potencia baja y predominio de lecturas observacionales; la ausencia de normalidad obliga a métodos no paramétricos y de remuestreo; en algunas celdas (modelos abiertos con pocas réplicas válidas) las medias se calculan sobre $n < 3$ y se marcan como tales; y, dado que OQuaRE agrega varias métricas, sus subcomponentes no se interpretan como pruebas independientes. Estas limitaciones no invalidan las afirmaciones centrales —de efecto grande y sostenido—, pero acotan el resto a tendencias observacionales.

Anexo D. Plantillas de prompts utilizadas

Este anexo reproduce las plantillas de prompts utilizadas en las cinco estrategias de generación. Las variables entre llaves se sustituyen en tiempo de ejecución por los valores correspondientes a cada (BD, ejecución). La descripción funcional de cada estrategia está en el apartado 3.4.

D.1. System prompt común (E1–E3)

"You are an expert ontologist specializing in biological knowledge graphs and semantic web technologies. Your expertise includes OWL 2, RDF, SPARQL, and biological domain ontologies (SO, GO, BioLink Model, SKOS). Your task is to analyze biological database samples and generate formal OWL ontology schemas in Turtle serialization. Domain context: The databases contain information about cis-regulatory modules (CRM), specifically enhancer sequences that modulate the transcription rate of their target genes in the human genome."

D.2. Indicación E1 — Zero-shot

Analyze the following biological database sample and generate a formal OWL ontology schema in Turtle format. DATABASE: {db_name}. DATABASE SAMPLE: {db_sample}. Think step by step: (1) Identify the main concepts/classes; (2) Identify relationships and attributes; (3) Map to standard ontologies (SO, GO, BioLink) where possible; (4) Generate the formal Turtle schema. Use prefixes rdf:, rdfs:, owl:, skos:, xsd:, obo:. Output ONLY valid Turtle code, no explanations.

D.3. Indicación E2 — Vocabulario controlado

[Prompt de D.2] + CONTROLLED VOCABULARY: {vocab_terms}. Use the provided vocabulary to name and align your ontology terms. Declare skos:exactMatch / skos:closeMatch where applicable. La variable {vocab_terms} se sustituye en tiempo de ejecución por la lista de vocabulario controlado extraída automáticamente de los esquemas cisreg (crm, crm2gene, crm2phen, crm2tfac) mediante la función load_vocabulary() de scripts/run_gpt_experiments.py, que recopila las líneas skos:prefLabel y skos:definition, deduplica y aplica un tope máximo de 80 términos. En el banco ejecutado, la materialización efectiva contiene 22 términos únicos y se reproduce literalmente en el Anexo F.

D.4. Indicación E3 — RAG semántico real (annotationRAG)

[E1 prompt] + SEMANTIC CONTEXT (retrieved via vector similarity): {retrieved_classes_with_scores}\n{retrieved_properties_with_scores}\n{relevant_fragments}. Use this structured metadata to construct a rich ontology. Prefer reusing the provided class names and properties when applicable.

D.5. Indicación E4 — OntoGenix

Tres plantillas externas en OntoGenix/GUI/: LlmPlanner.prompt (analiza CSV y devuelve JSON con clases, propiedades y jerarquía), LlmOntology.prompt (construye TBox a partir del plan), LlmOntoMapper.prompt (mapea filas del CSV a instancias). Detalle completo en el repositorio OntoGenix original (Val-Calvo et al., 2025): <https://github.com/tecnomod-um/OntoGenix>.

Anexo E. Corpus de preguntas de competencia

Las 15 preguntas de competencia (competency questions, CQ; Grüninger y Fox, 1995) del corpus cisreg utilizado en el apartado 4.3.3 se reproducen a continuación, agrupadas por dimensión del modelado. Cada pregunta se acompaña de un patrón de requisitos TBox (conjunto mínimo de clases y predicados que la ontología debe declarar para que la consulta sea ejecutable en principio).

CQ01 • Localización genómica. *¿En qué cromosoma se localiza el módulo regulador X?*

Requisitos TBox: Clases: CRM; Predicados: hasChromosome, hasStart, hasEnd.

CQ02 • Regulación gen → CRM. *¿Qué genes regula el CRM X?*

Requisitos TBox: Clases: CRM, Gene; Predicados: regulates / targets.

CQ03 • Unión TF. *¿Qué factores de transcripción se unen al CRM X?*

Requisitos TBox: Clases: CRM, TranscriptionFactor; Predicados: bindsTo / hasFactor.

CQ04 • Asociación a enfermedad. *¿Qué enhancers están asociados a la enfermedad Y?*

Requisitos TBox: Clases: Enhancer, Disease; Predicados: associatedWith / hasDisease.

CQ05 • Evidencia experimental. *¿Qué tipo de evidencia experimental soporta la asociación CRM–gen X–Y?*

Requisitos TBox: Clases: Evidence, ExperimentMethod; Predicados: hasEvidence, derivedFrom.

CQ06 • Contexto biológico. *¿En qué tejido o tipo celular está activo el CRM X?*

Requisitos TBox: Clases: CRM, BioSample; Predicados: activeIn / hasBiosample.

CQ07 • Regulación inversa. *¿Qué CRMs regulan el gen X?*

Requisitos TBox: Inversión de CQ02. Predicados: regulatedBy.

CQ08 • Procedencia. *¿De qué base de datos procede el CRM X?*

Requisitos TBox: Clases: CRM; Predicados: providedBy, source.

CQ09 • Coordenadas. *¿Cuál es la longitud genómica del CRM X (end – start)?*

Requisitos TBox: Clases: CRM; Predicados: hasStart, hasEnd; xsd:integer.

CQ10 • Referencia bibliográfica. *¿Qué publicaciones PubMed soportan la asociación CRM–enfermedad X–Y?*

Requisitos TBox: Clases: Evidence, Publication; Predicados: pmid / citedIn.

CQ11 • Tipo de regulación. *¿Es el CRM X un enhancer convencional o un super-enhancer?*

Requisitos TBox: Clases: Enhancer, SuperEnhancer; Predicados: typeOf.

CQ12 • Score de confianza. *¿Cuál es el score de confianza del CRM X?*

Requisitos TBox: Clases: CRM; Predicados: hasScore; xsd:float.

CQ13 • Método experimental. *¿Qué método experimental se empleó para detectar el CRM X?*

Requisitos TBox: Clases: ExperimentMethod; Predicados: detectedBy.

CQ14 • Asociación a mutación. *¿Qué mutaciones se han descrito en el CRM X?*

Requisitos TBox: Clases: Mutation; Predicados: hasMutation.

CQ15 · Regulación de múltiples genes. ¿Cuáles son los CRMs que regulan más de un gen?

Requisitos TBox: Clases: CRM, Gene; Predicados: regulates, count > 1.

Una pregunta se considera satisfecha si la ontología cubre al menos el 50 % de las clases requeridas y el 50 % de los predicados requeridos. La métrica reportada es la cobertura (n CQ satisfechas / 15). La evaluación se ejecuta automáticamente con scripts/competency_questions.py.

Anexo F. Vocabulario controlado utilizado en E2 ({vocab_terms})

Este anexo reproduce la lista literal de términos que la función load_vocabulary() (scripts/run_gpt_experiments.py) inyecta en la variable {vocab_terms} de la plantilla D.3. El procedimiento recorre los cuatro esquemas cisreg de referencia (crm.txt, crm2gene.txt, crm2phen.txt, crm2tfac.txt en data/samples/schemas/), extrae las líneas que contienen skos:prefLabel o skos:definition, elimina duplicados por igualdad de cadena y limita el resultado a los primeros 80 elementos. El nombre original del marcador ({vocab_list_80_terms}) hace referencia a ese tope superior; en el banco ejecutado, la materialización efectiva contiene 22 términos únicos, que se listan a continuación en el orden en que se envían al modelo. Los términos con prefijo bgw! son URIs relativas del namespace BioGateway.

F.1. Lista literal de términos (22 entradas)

```
01. skos:definition "Cis-regulatory module located in Homo sapiens chr_name between
start_position and end_position" ;
02. skos:prefLabel "crm/crm_ID" ;
03. skos:definition "Cis-regulatory module located in Homo sapiens chr_name between
start_position and end_position according to database_name" ;
04. skos:prefLabel "crm#crm_ID#database_name" ;
05. skos:prefLabel "CRM" ;
06. skos:prefLabel "chr_name" ;
07. skos:prefLabel "GRCh38" ;
08. skos:prefLabel "database_name" ;
09. skos:prefLabel "Database" .
10. skos:prefLabel "bgw!crm_ID--hgncsymbol!9606/gene_sybmol" ;
11. skos:definition "Association between cis-regulatory module CRMHS00000003515 and gene
TBX5" .
12. skos:prefLabel "bgw!crm_ID--hgncsymbol!9606/gene_symbol#database_name" ;
13. skos:definition "Association between cis-regulatory module crm_ID and gene gene_symbol
according to database_name" ;
14. skos:prefLabel "bgw!crm_ID--doid!doid" ;
```

15. skos:definition "Association between cis-regulatory module crm_ID and disease DOID:doid"
;
16. skos:prefLabel "bgw!crm_ID--doid!doid#database_name" ;
17. skos:definition "Association between cis-regulatory module crm_ID and disease DOID:doid
according to database_name" ;
18. skos:prefLabel "bgw!crm_ID--uniprot!uniprot_ID" ;
19. skos:definition "Association between cis-regulatory module crm_ID and transcription
factor uniprot_ID" ;
20. skos:prefLabel "Transcription Factor" ;
21. skos:prefLabel "bgw!crm_ID--uniprot!uniprot_ID#database_name" ;
22. skos:definition "Association between cis-regulatory module crm_ID and transcription
factor uniprot_ID according to database_name" ;

F.2. Notas sobre la materialización de la lista

Tres observaciones sobre la lista anterior. Primera, el tope de 80 términos nunca se alcanzó: los cuatro esquemas cisreg incluidos aportan sólo 22 pares prefLabel/definition únicos tras la deduplicación por cadena exacta. Segunda, la lista mezcla etiquetas SKOS (prefLabel) y definiciones en lenguaje natural (definition) porque load_vocabulary() no distingue entre ambas: el modelo recibe así tanto el nombre canónico como su glosa, en el formato Turtle original de los esquemas. Tercera, algunos elementos son URIs relativas del namespace BioGateway (bgw!crm_ID--...) más que términos canónicos reutilizables; su presencia en la lista es un artefacto del procedimiento de extracción y no una decisión de diseño. Estas tres características —tamaño reducido, mezcla de etiquetas y definiciones, y presencia de URIs relativas— son relevantes para interpretar el rendimiento de E2 reportado en 4.3.1 y la fidelidad léxica discutida en 4.3.2.